MA124 Maths by Computer

Project: Constructing and applying Machine Learning models

Background

Machine Learning is the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyse and draw inferences from patterns in data. In machine learning we are interested in creating functions or models which fit real data. Such functions can be found by making them dependent on a number of parameters which are updated over and over again to improve the fit to the data. You will explore in this project.

Suggested reading

There are many great articles, books, videos about machine learning. This list just picks out a few:

Linear Regression Model. Short, simple article covering a task similar to A2 below.

Logistic Regression Model An article which covers models similar to those in A3 and A4 below.

What is a neural network. Great YouTube series of introductory videos about training neural networks by the excellent 3Blue1Brown, well worth watching.

Blog: An Introduction to Neural Networks. Good discussion of the basic ideas used in this project.

Hands on Machine Learning with Python This might accompany a full course on machine learning. It has much more detail that you will need. As the title says, it's all done using Python. It's well-written and popular.

Struture of project

There are 6 tasks in the document below, tasks A1-A4 in Section A and then tasks B1 and B2 in section B. Your group should do all the tasks in section A and at least one of the two tasks in section B. A1-A4 are worth approximately 60% of the credit for this submission and section B is worth approximately the other 40%.

Notes on submission

Read through the document **MA124 Maths by Computer Tutor Group Projects Information for Students.pdf** on the MA124 Moodle page.

Before submitting see the notes at the end of this document.

Allowed libraries for this project:

numpy matplotlib copy pandas scikitlearn. **Note you may not use scikitlearn in any of tasks A2, A3 or A4.**

This means that you may include these lines of code at the start of of your code cells (and please use the aliases given below).

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import copy
import sklearn # Not to be used in any of tasks A2, A3 or A4.
from numpy.matrixlib.defmatrix import matrix
```

Section A - (worth approximately 60% of the marks)

Task A1 - Linear Regression machine learning task using SciKitLearn (worth approximately 20% of the marks)

In a recent research article published in the journal Computer Communications, authors Sathishkumar V E, Jangwoo Park, and Yongyun Cho sought to predict the "bike count required at each hour for the stable supply of rental bikes"[1]. They employed several regression models, including linear regression. The dataset used in the original study is available here.

Task: Apply machine learning to a modified version of the original dataset and report the results.

[1] Sathishkumar V E, Jangwoo Park, and Yongyun Cho. 'Using data mining techniques for bike sharing demand prediction in metropolitan city.' Computer Communications, Vol.153, pp.353-366, March, 2020 web link.

The original research article and a modified dataset are posted on the MA124 Moodle page in the resources for this project. You will need to refer to the article for some of the tasks below. You will need to download SeoulBikeData_mod.csv and put it into the folder with this file.

SeoulBikeData_mod.csv has been modified from the original dataset to remove the categorical variables, and to convert dates to months. Months have been coded by number, e.g. 1 = January, etc. Only half the months are included in the modified dataset.

While the number of tasks is large, this is in part because the instructions are rather specific. Many of them are very similar to things you saw in term 1, in the week 8 and week 10 notebooks and lectures.

Specifically, your code should

1. Import needed libraries. (You will need pandas, as well as things from sklearn, and of course numpy and matplotlib.)

2. Using pandas, read SeoulBikeData_mod.csv into a Dataframe.

3. `describe` the Dataframe.

4. Plot a histogram of `Rented Bike Count`. Do not plot this as a density, but as a count. See Fig. 3 of the article. The vertical axis in the article is labelled "frequency", but is the same as the count.

   Produce a box plot similar to that in Fig. 3 of the article.

   Try to generate both the histogram and box plot to look approximately as they do in the article.

5. From the full Dataframe, create a new Dataframe `X` containing all the columns except `Rented Bike Count` and a Series `y` containing only the `Rented Bike Count` column. These are your design matrix and target respectively.

6. Perform a test-train split to create `X_train`, `X_test`, `y_train` and `y_test`. You **must** use the same percentage of data for testing and training as was used in the article and you **must** state what they are. You can find these in the article.

7. Create and train a linear regression model.

8. Use the trained model to obtain `y_pred`, the prediction on the test data `X_test`. Form the residual `resid = y_test - y_pred`.

9. Compute and report: Rsquared (R2), the Root Mean Squared Error (RMSE), the Mean Absolute Error (MAE), and the Coefficient of Variation (CV). Compare these results to those on the top, right of Table 4 of the article. (Note, the modified dataset we are studying is different from that used in the article. Hence the results will not be identical. However, the procedure is very close to that used in the article.)

10. Produce and comment on the following plots.

- Histograms of `y_test` and of `y_pred` (on the same plot). These should be reported as counts rather than densities.

- A scatter plot of `resid` as a function of `y_test` corresponding to Fig. 9 of the article. (Recall what `y_test` represents and label the plot appropriately.) Unlike Fig. 9 of the paper, you should use a colormap to plot the different `Hours` in different colours.

- A scatter plot of `resid` as a function of `X_test['Month']`. Use a colormap to indicate the absolute value of `resid`.

- A scatter plot of `resid` as a function of `X_test['Rainfall(mm)']`. Use a colormap to indicate the absolute value of `resid`.

Insert code and markdown cells below in which to answer this.

In [2]:
```
from sklearn import linear_model
from sklearn import datasets
from IPython.display import display, Math
import math
from sklearn.model_selection import train_test_split
```

In [3]:
```
bikes = pd.read_csv("SeoulBikeData_mod.csv") # Using pandas to import data frame
bikes.head() # Printing first 5 rows of the data frame
```

Out[3]:

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Ra (M
0	206	0	-3.2	40	0.5	1358	-14.9	
1	230	1	-3.3	41	0.7	2000	-14.7	
2	178	2	-3.7	42	0.9	2000	-14.7	
3	119	3	-4.0	44	1.0	2000	-14.4	
4	62	4	-4.2	53	1.1	1994	-12.3	

In [4]:
```
bikes.describe()
```

Out[4]:

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibilit (10m
count	4416.000000	4416.00000	4416.000000	4416.000000	4416.000000	4416.00000
mean	704.591259	11.50000	13.203986	59.257246	1.653193	1398.64130
std	637.407244	6.92297	11.529724	20.162981	0.976361	621.56053
min	0.000000	0.00000	-17.800000	0.000000	0.000000	27.00000
25%	189.000000	5.75000	4.500000	44.000000	0.900000	853.00000
50%	542.000000	11.50000	14.300000	58.000000	1.500000	1621.00000
75%	1047.000000	17.25000	22.200000	75.000000	2.200000	1999.00000
max	3298.000000	23.00000	38.000000	98.000000	6.700000	2000.00000

In [5]:
```
bikes.shape # Finding number of rows and columns
```

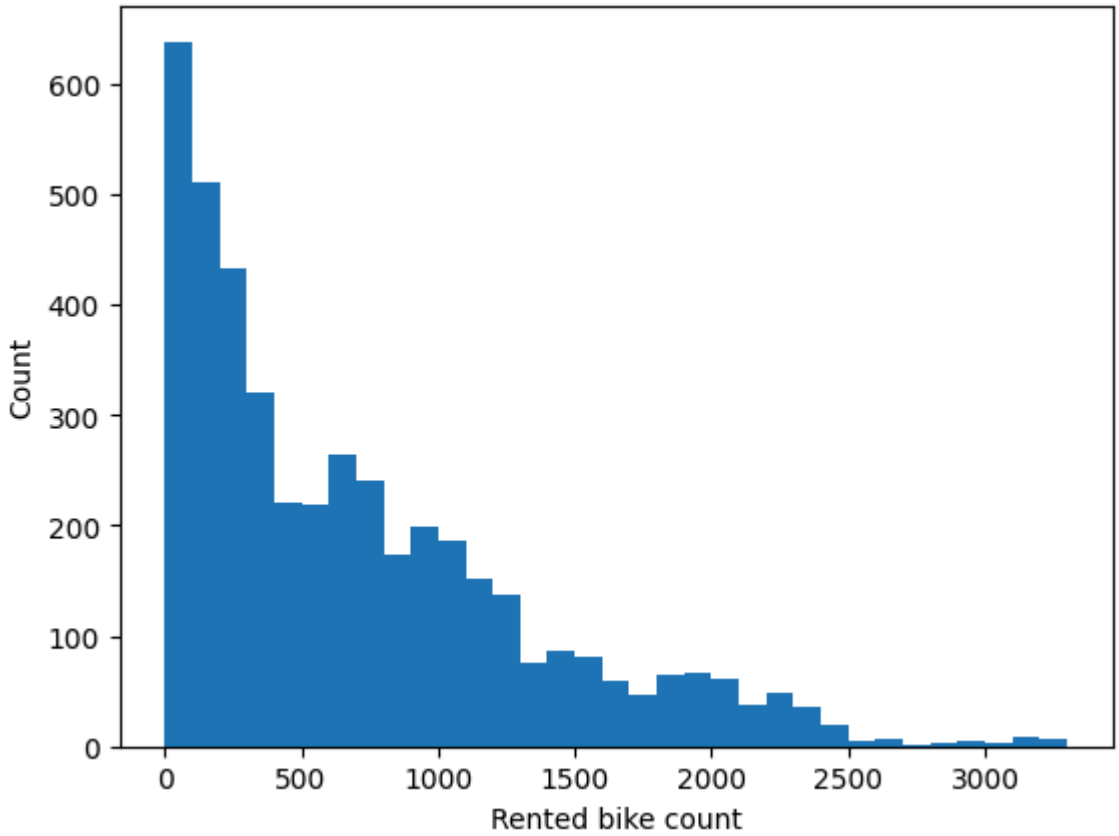
Out[5]: (4416, 11)

The data frame consists of 4416 rows and 11 columns. It has the information of the number of rented bikes each hour, with the hour and month recorded for each entry. Each entry also has various weather conditions recorded for that hour.

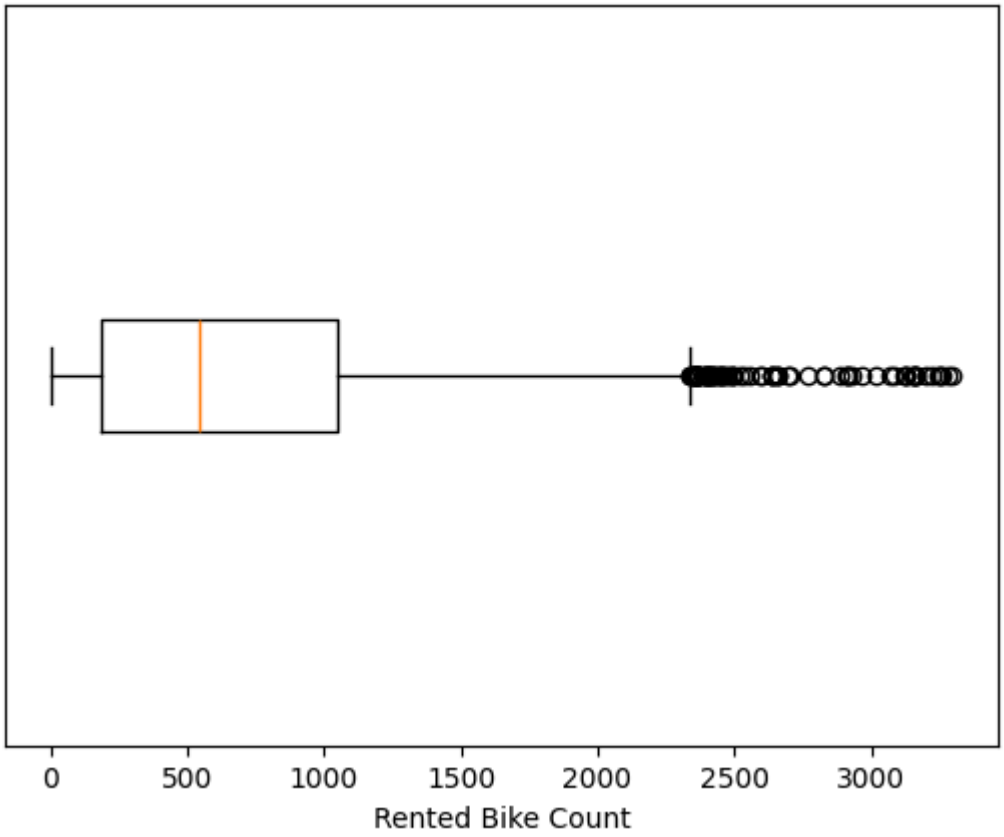
In [6]:
```
bike_counts = bikes["Rented Bike Count"] # Extracting values from the column wit
max(bike_counts) # We find the max count is 3298. So, we will use 33 buckets of
```

Out[6]: 3298

In [7]:
```
bin_edges = [0] * 34 # Creating list of 34 values
for i in range(34):
    bin_edges[i] = i*100  # Filling in the list of values with multiples of 100
plt.hist(bike_counts, bins=bin_edges) # Creating histogram of bike counts, using
plt.xlabel("Rented bike count")
plt.ylabel("Count")
plt.show()
```



In [8]:
```
plt.boxplot(bike_counts, vert = False) # Using matplotlib to create boxplot and
plt.xlabel("Rented Bike Count")
plt.yticks([]) # Removing errant label from y-axis
plt.show()
```



In [9]:
```
X = bikes.drop("Rented Bike Count",axis=1) # Using .drop to remove the column la
y = bikes["Rented Bike Count"]
```

We use 75% of the data for training, and 25% of the data for testing. This is because a larger part of the data is usually needed to teach machine learning models. Thus, we use 3312 entries for training, and 1104 entries for testing. Because the data frame is currently sorted in chronological order, and the date and time is one of the variables in the X data frame, we will select these entries at random instead of taking the first 3312 for training.

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random
# into SKlearn. test_size reflects that we want 25% of the data to be for testin
# shuffle = True makes the sampling random instead of just being the first 25% o
regr = linear_model.LinearRegression()
regr.fit(X_train,y_train)
y_pred = regr.predict(X_test)
resid=(y_test-y_pred)
```

Report of accuracy statistics

In [11]:
```
n = 1104 # n is the number of sets of values in the test data
y_mean = y_test.mean()
R2 = 1 - (sum((resid)**2))/(sum((y_test-y_mean)**2)) #R^2 calculation from the a
display(Math(rf"R^2 = {R2:.6f}")) # LaTeX display
```

$R^2 = 0.523237$

For this model, $R^2 = 0.5232$. The value of R^2 from the article was 0.55. So, the value from this model is less than 5% below the value from the article

In [12]:
```
RMSE = math.sqrt((sum((resid)**2))/n) #Root mean squared error calculation from
display(Math(rf"RMSE = {RMSE:.3f}")) # LaTeX display
```

$RMSE = 442.673$

For this model, root mean squared error was 442.67. The value of the RMSE in the article was 427.71. So, the value from this model is less than 3.5% above the value from the article.

In [13]:
```
MAE = sum(abs(resid))/n #Mean absolute error calculation from the article
display(Math(rf"MAE = {MAE:.3f}")) # LaTeX display
```

$MAE = 325.469$

For this model, the mean absolute error was 325.469. The value of the MAE in the article was 322.32. So, the value from this model is less than 1% above the value from the article

In [14]:
```
CV = math.sqrt((sum((resid)**2))/n)/y_mean*100 #Coefficient of variation calcula
display(Math(rf"CV = {CV:.3f}")) # LaTeX display
```

$CV = 62.207$

For this model, the coefficient of variation was 62.207. The value of the CV in the article was 61.03. So, the value from this model is less than 2% above the value from the article

Overall, we can see that the results obtained by our model were very similar in quality to that of the model, never deviating by more than 5%.

Plots of the data

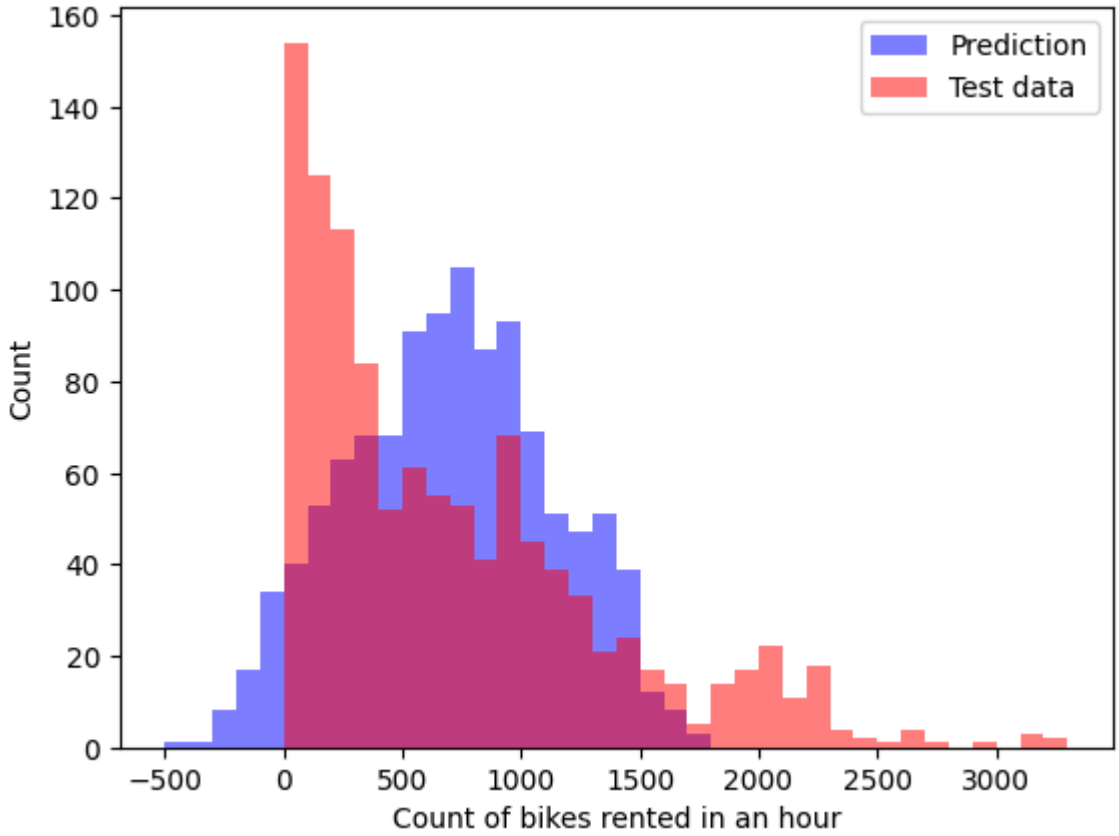
In [15]:
```
# Finding minimum, maximum, and mean of test data and prediction data to inform
print("The maximum value of y_test is:",  max(y_test))
print("The minimum value of y_test is:", min(y_test))
print("The maximum value of y_pred is:", int(max(y_pred)))
print("The minimum value of y_pred is:", int(min(y_pred)))
print("The mean of y_test is:",y_test.mean())
print("The mean of y_pred is:", y_pred.mean())
```

```
The maximum value of y_test is: 3298
The minimum value of y_test is: 0
The maximum value of y_pred is: 1714
The minimum value of y_pred is: -460
The mean of y_test is: 711.616847826087
The mean of y_pred is: 703.9085295135741
```

The minimum and maximum values of y_pred and y_test are -460 and 3298. So, we will use 38 buckets of width 100, from -500 to 3300

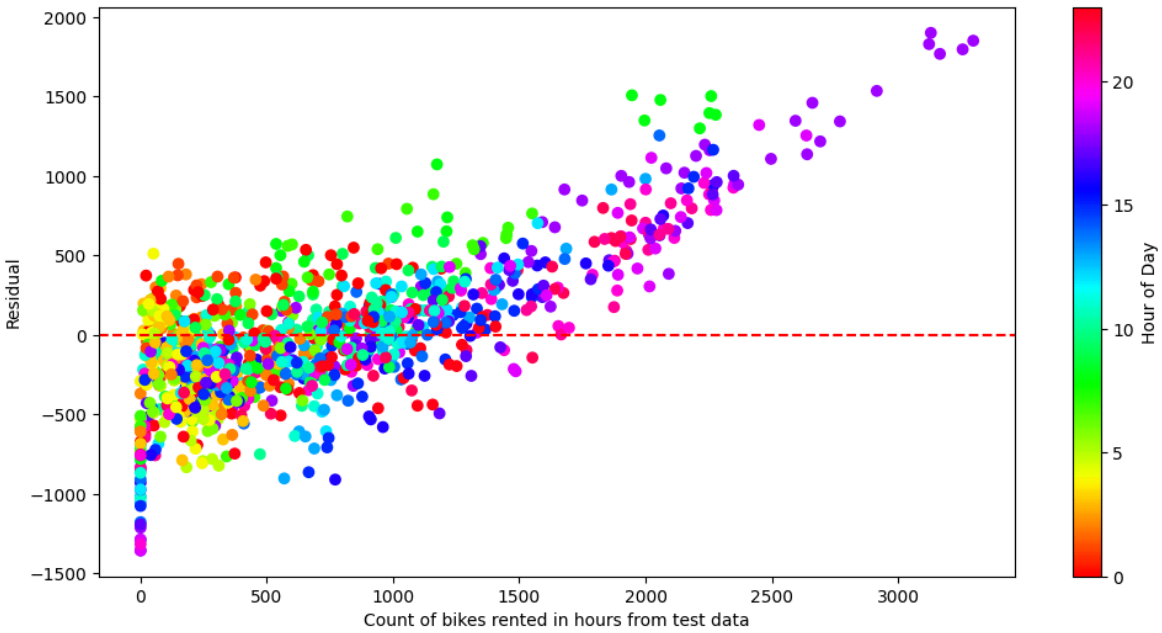
In [16]:
```
bin_edges2 = [0] * 39 # Creating list of 39 values
for i in range(-5,34,1):
    bin_edges2[i+5] = i*100  # Filling in list with multiples of 100 from -500 t
plt.hist(y_pred, alpha=0.5, bins=bin_edges2, color = "blue", label = "Prediction
plt.hist(y_test, alpha=0.5, bins=bin_edges2, color = "red", label = "Test data")
# Above, alpha=0.5 makes the histograms translucent to make observations easier
```

```
plt.xlabel("Count of bikes rented in an hour")
plt.ylabel("Count")
plt.legend()
plt.show()
```



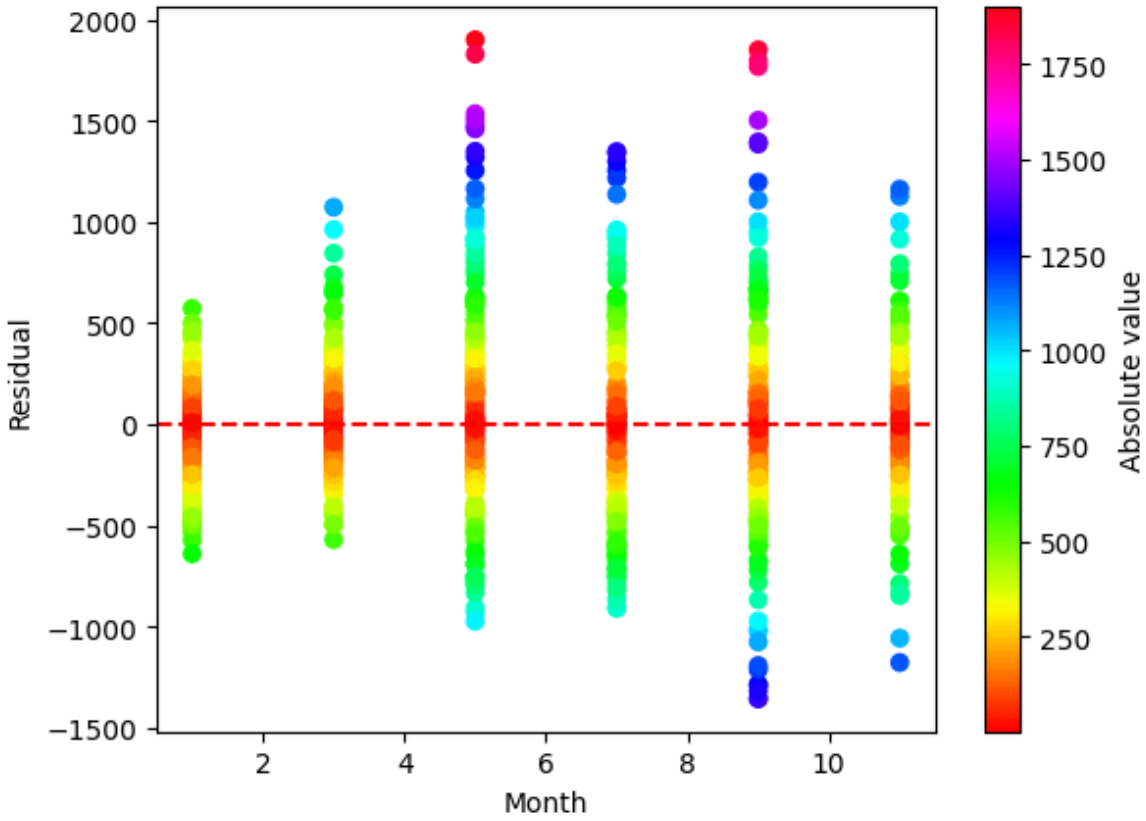
The prediction generally fails to account for extreme values. Some test values are over 3000, while the model never predicts a value above 1800. Also, the test data has a high cluster of values at low positive values, which is not reflected in the prediction. Also, the model sometimes predicts negative values, which is impossible for a count of bikes rentals. However, the mean of the prediction data is very accurate to the mean of the test data.

In [17]:
```
plt.figure(figsize=(12, 6)) # Controlling size of plot
plt.axhline(y=0, linestyle='--', color='red') # Horizontal line to show where th
plt.scatter(y_test, resid, c = X_test["Hour"], cmap="hsv") # Creating scatter pl
plt.colorbar(label='Hour of Day')
plt.xlabel("Count of bikes rented in hours from test data")
plt.ylabel("Residual")
plt.show()
```



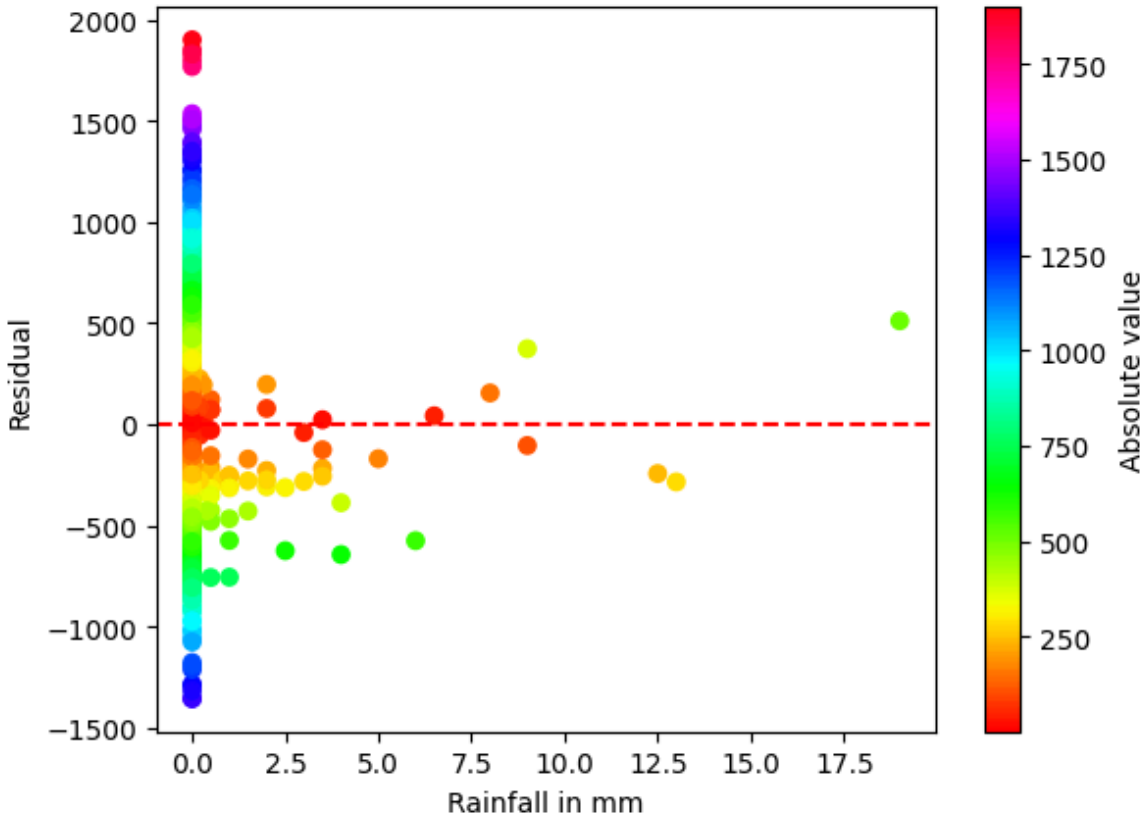
The scatter plot shows that the model performs better (the residual is smaller) when the actual bike count is small, but non-zero. We see that, when the bike count is zero, the residual tends to be very negative. When the bike count is high, the residual is consistently very high. The absolute value of the residual is minimised when the bike count is roughly between 500 and 1000. The colouring seems to show that the bike count is higher at particular hours. Rentals seems to peak at around hour 17, with a smaller peak at around hour 7. This makes sense, as these times would likely be busy periods when people go to work / come home from work. We also see that certains hours have extremely low bike counts. For example, hours 3-4 have very low bike counts, which makes sense, as most people would be asleep at this point.

In [18]:
```
plt.axhline(y=0, linestyle='--', color='red') # Horizontal line to show where th
plt.scatter(X_test["Month"], resid, c = abs(resid), cmap="hsv") # Creating scatt
plt.colorbar(label='Absolute value')
plt.xlabel("Month")
plt.ylabel("Residual")
plt.show()
```

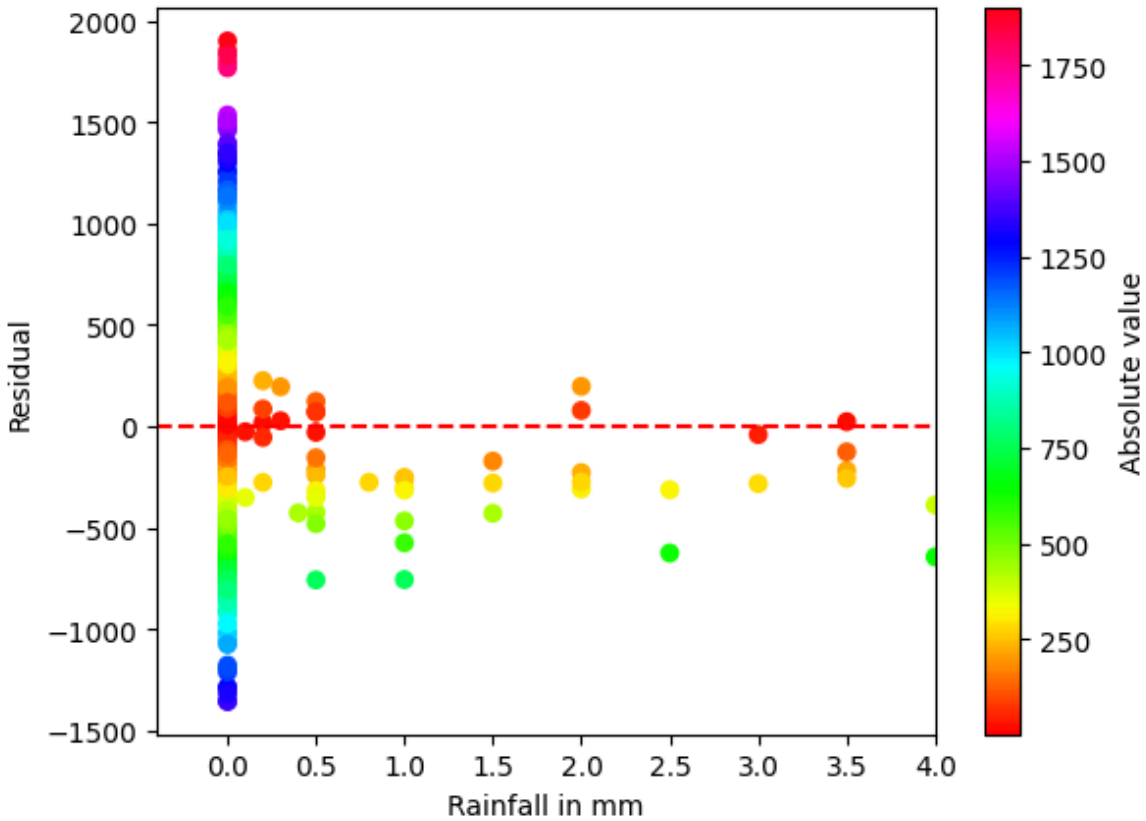


The plot shows that the average absolute value of the residual is smallest in months 1, 3, and 11. This implies that there are fewer bike rentals during these months, based off of the positive correlation between residual and number of bikes rented from the previous scatter graph. The average absolute value of the residual is then highest in months 5, 7, and 9. This makes sense, since Summer occurs in Seoul roughly from June to August, which is months 6 to 8. During this time, people are probably more likely to rent bikes.

In [19]:
```
# Here, we do not limit the x-axis to show all values, including outliers
plt.axhline(y=0, linestyle='--', color='red') # Horizontal line to show where th
plt.scatter(X_test['Rainfall(mm)'], resid, c = abs(resid), cmap="hsv") # Creatin
plt.colorbar(label='Absolute value')
plt.xlabel("Rainfall in mm")
plt.ylabel("Residual")
plt.show()
```



In [20]:
```
# Here, we limit the x-axis to make observations easier
plt.axhline(y=0, linestyle='--', color='red') # Horizontal line to show where th
plt.scatter(X_test['Rainfall(mm)'], resid, c = abs(resid), cmap="hsv") # Creatin
plt.colorbar(label='Absolute value')
plt.xlabel("Rainfall in mm")
plt.ylabel("Residual")
plt.xlim(-0.4,4)
plt.show()
```



The scatter plot shows that a vast majority of the hours have 0 millimetres of rainfall. The average of the absolute value of the residual is quite large for 0 rainfall. The absolute value of the residual seems to decrease for higher values of rainfall, but there are too few data points to make a definitive conclusion for this.

Overall, based off of the above results, especially how large the residual tends to be relative to the actual bike count, I think that a linear regression model is not suitable for the data frame. This mirrors the results from the article, in which the linear model performed the worst.

Task A2 Linear Regression, on a very small dataset, from scratch with only numpy (worth approximately 10% of the marks)

(illustrating idea of rates of change with respect to the model parameters and gradient descent)

Consider a really small dataset consisting of only three points in the plane: $(1,1),(2,4),(3,5)$.

For each point (training example) consider the x-coordinate as a single input feature, and the y-coordinate as the output or label of the example. The objective is to find the best line to fit these data. Here 'best' will mean the line which minimises the mean of the sum of the squares of the residuals with respect to the three points.

In this task and in the following tasks A3 and A4 we will use a slightly different convention to store our data with respect to the one used in A1 or the lectures. X_{train} and y_{train} will now be the transpose of the corresponding matrices in A1. In A3 and A4 this will be a more convenient form for applying functions given by matrices to the data with the matrix on the left of the data matrix.

The feature matrix X_{train} contains the feature vectors for all the training examples. More precisely $X_{\text{train}} = \begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$ has one column for each training example and each column is the feature vector for that training example (in this case we have only one feature which is the x-coordinate).

The labels matrix y_{train} contains the labels for all the training examples. More precisely $y_{\text{train}} = \begin{pmatrix} 1 & 4 & 5 \end{pmatrix}$ has one column for each training example and each column contains the label for that training example (the y-coordinate).

The model here is a simple linear model

$$\hat{y} = mx + c$$

with trainable parameters m and c. The goal is to find the optimal values of m and c to fit these training data. In the Machine Learning context this means defining a loss function between prediction and true label:

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

and a cost function which is simply the average on all training examples of these losses.

Specifically, you should

1. Show, in a markdown cell, how the cost J depends only on the trainable parameters m and c, and can be computed to be:

$$J = \frac{14m^2 + 12cm - 48m + 3c^2 - 20c + 42}{3}$$

2. Find, in a markdown cell, expressions for the partial derivatives $\frac{\partial J}{\partial m}$ and $\frac{\partial J}{\partial c}$.

3. Write a function `model1(alpha, num_iterations)` which takes as inputs a learning rate `alpha` and a number of iterations `num_iterations`, and returns optimized values of m and c through Gradient Descent. More precisely the function should initialize m and c randomly between -2 and 2 and then perform `num_iterations` steps of gradient descent with learning rate `alpha`. This means that the value of m is updated to $m - \frac{\partial J}{\partial m}\alpha$ and the value of c is updated to $c - \frac{\partial J}{\partial c}\alpha$ in each iteration. Your function should print the cost J periodically throughout the iteration process. You may wish to refer to the Machine learning project lecture for help with this. You may wish to build helper functions for the various tasks you need this function to do and refer to them.

4. Plot the 3 datapoints along with the optimal line $mx + c$ your model has found.

5. Explore the effect of changing the learning rate and the number of iterations.

Insert code and markdown cells here in which to answer this task

In [21]:
```
Xtrain = np.array([1,2,3])
ytrain = np.array([1,4,5])
```

In [22]:
```
def cost(m,c):
    J = (Xtrain*m)**2 + 2*Xtrain*m*c - 2*ytrain*c - 2*Xtrain*ytrain*m + ytrain**
    mean = np.mean(J)
    return(mean)
```

$$J = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

Substituting $\hat{y}_i = mx_i + c$ and expanding gives:

$$J = \frac{\sum x_i^2 m^2 + 2x_i mc + c^2 - 2y_i c - 2x_i y_i m + y_i^2}{n}$$

Where n is the amount of training points

In [23]:
```
def Jm(m,c):
    J = 2*m*Xtrain**2 + 2*Xtrain*c - 2*Xtrain*ytrain
    mean = np.mean(J)
    return(mean)
```

```
def Jc(m,c):
    J = 2*Xtrain*m - 2*ytrain + 2*c
    mean = np.mean(J)
    return(mean)
```

$$\frac{\partial J}{\partial m} = \frac{\sum 2mx_i^2 + 2x_i c - 2x_i y_i}{n}$$

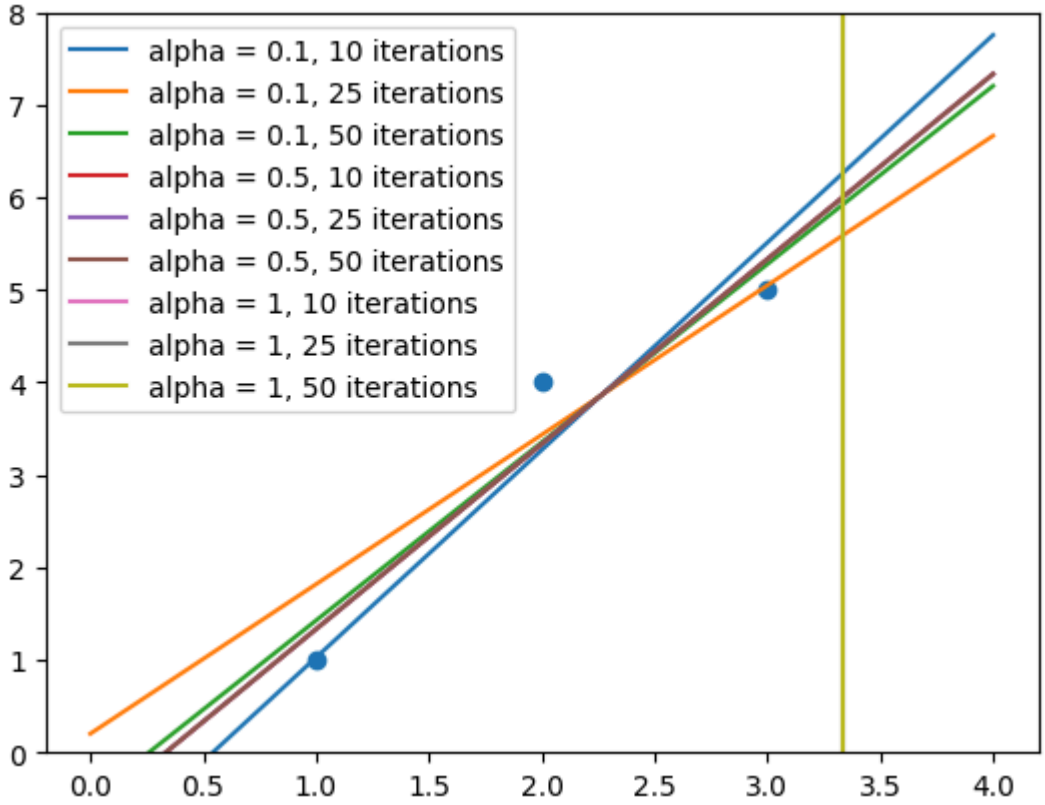
$$\frac{\partial J}{\partial c} = \frac{\sum 2x_i m - 2y_i + 2c}{n}$$

In [24]:
```
def model1(alpha,num_iterations):
    m = np.random.uniform(-2,2)
    c = np.random.uniform(-2,2)
    print("\nLearning Rate:",alpha, "Iterations:",num_iterations)
    #print(" \nInitial cost =", cost(m,c))

    for i in range(1, num_iterations+1): #steps of gradient descent
        m -= Jm(m,c) * alpha
        c -= Jc(m,c) * alpha
        if (i % 5 == 0): #prints every 5th cost
            print("Cost", i,"=", cost(m,c))

    print("Final Parameters: m =",m," c =",c)
    return(m,c)
```

In [25]:
```
#Plotting the graph
learn_rates = [0.1, 0.5, 1]
iterations = [10, 25, 50]

X = np.linspace(0,4,50)

plt.scatter(Xtrain, ytrain)

for alpha in learn_rates:
    for num_iterations in iterations:
        parameters = model1(alpha, num_iterations)
        m = parameters[0]
        c = parameters[1]

        line = m*X + c
        label = ('alpha = ' +str(alpha)+ ', ' +str(num_iterations)+ ' iterations

        plt.plot(X,line, label=label) #plots a line on the same axes for each pa

plt.ylim(0,8)
plt.legend()
plt.show()
```

```
Learning Rate: 0.1 Iterations: 10
Cost 5 = 0.27997725288994624
Cost 10 = 0.2656023976019753
Final Parameters: m = 2.24282141226407  c = -1.2161218453576714

Learning Rate: 0.1 Iterations: 25
Cost 5 = 0.5638419947855923
Cost 10 = 0.47881484411727254
Cost 15 = 0.4149504515794369
Cost 20 = 0.366981530252364
Cost 25 = 0.3309517909587535
Final Parameters: m = 1.6155718677010344  c = 0.20321554905909978

Learning Rate: 0.1 Iterations: 50
Cost 5 = 0.2753339989299906
Cost 10 = 0.26211472751420545
Cost 15 = 0.25218571787052885
Cost 20 = 0.2447279802561114
Cost 25 = 0.23912642961594602
Cost 30 = 0.23491907194306894
Cost 35 = 0.23175890081820405
Cost 40 = 0.22938527767634262
Cost 45 = 0.22760243535642724
Cost 50 = 0.22626333199093673
Final Parameters: m = 1.9258874677040796  c = -0.4989651682849468

Learning Rate: 0.5 Iterations: 10
Cost 5 = 0.23925021955149595
Cost 10 = 0.222222510592851
Final Parameters: m = 2.0006576898546884  c = -0.6679820463760434

Learning Rate: 0.5 Iterations: 25
Cost 5 = 0.2336549712875682
Cost 10 = 0.22222241583683167
Cost 15 = 0.2222222222255014
Cost 20 = 0.2222222222222209
Cost 25 = 0.22222222222222535
Final Parameters: m = 2.000000000037558  c = -0.6666666667417828

Learning Rate: 0.5 Iterations: 50
Cost 5 = 0.22446749364534246
Cost 10 = 0.2222222602460903
Cost 15 = 0.22222222222286692
Cost 20 = 0.22222222222222152
Cost 25 = 0.22222222222222107
Cost 30 = 0.22222222222222418
Cost 35 = 0.22222222222222146
Cost 40 = 0.22222222222222146
Cost 45 = 0.22222222222222146
Cost 50 = 0.22222222222222146
Final Parameters: m = 2.000000000000001  c = -0.6666666666666684

Learning Rate: 1 Iterations: 10
Cost 5 = 922186006.5093504
Cost 10 = 9061703082978640.0
Final Parameters: m = 60884986.500455946  c = -202951138.6123004

Learning Rate: 1 Iterations: 25
Cost 5 = 538545062.6328269
Cost 10 = 5293028740698989.0
```

```
Cost 15 = 5.169109488217398e+22
Cost 20 = 5.047959031581903e+29
Cost 25 = 4.9296474939774485e+36
Final Parameters: m = 1.4200964667253627e+18  c = -4.733654889086541e+18

Learning Rate: 1 Iterations: 50
Cost 5 = 344917940.02908534
Cost 10 = 3386680835008888.5
Cost 15 = 3.307379340372688e+22
Cost 20 = 3.22986293382087e+29
Cost 25 = 3.154163022501967e+36
Cost 30 = 3.0802373266668755e+43
Cost 35 = 3.00804426432314e+50
Cost 40 = 2.9375432268780736e+57
Cost 45 = 2.8686945574981175e+64
Cost 50 = 2.8014595288067526e+71
Final Parameters: m = 3.3853384253585996e+35  c = -1.1284461417861999e+36
```



For both learning rates 0.1 and 0.5 the cost is minimised to 0.22 whereas learning rate 1 increases cost to extreme large values which gives unsuitably large m and c values resulting in a very inaccurate line (cannot be seen with current graph limits). alpha = 0.5 gives a consistent cost and accurate m and c (we cannot see the lines due to the accuracy creating an overlap) from around the 10th iteration whereas alpha = 0.1 is still quite accurate but takes more iterations to refine and is very rarely accurate (even with 50 iterations).

Task A3 Machine Learning for binary classification using Logistic Regression (worth approximately 10% of the marks)

Here you will start by running some code provided to you which will create a dataset. You will use this to train a Logistic Regression classification model.

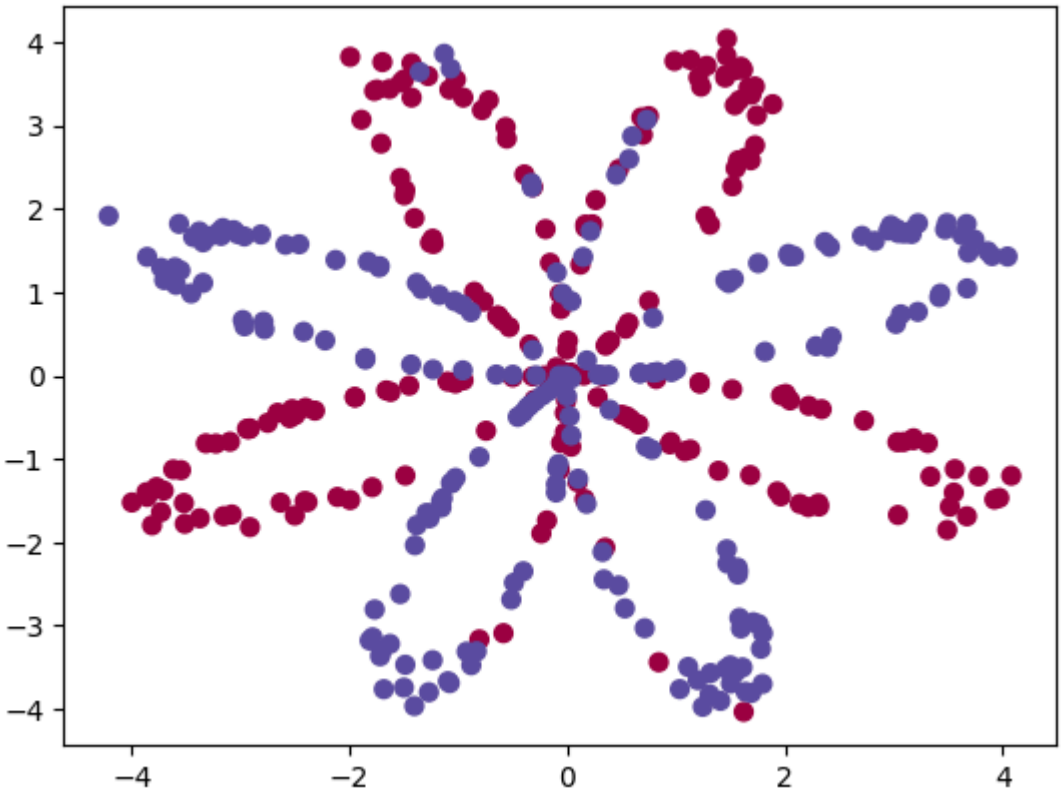
The dataset consists of a set of points (x_0, x_1) in the plane each of which is either blue or red. Your model will take as input the coordinates of a point in the plane and return 0 if the point is likely to be red, or 1 if the point is likely to be blue. In practice your model will output a float (real number) between 0 and 1 which is the probability for the point to be blue, and you will then use 0.5 as a threshold to predict the point as being blue or red.

You are guided through this task below. You will see code cells that you need to run and code cells that you need to edit/complete. You can also add new code/markdown cells as required. You may wish to add markdown cells to comment on your code or to comment on any outputs you see.

Create and visualise the dataset

Run the cell below to create a planar dataset. A visualisation of this is provided, you will see that it looks like a flower with some red points and some blue points.

In [26]:
```
# Create the flower dataset

def load_flower_dataset():
    np.random.seed(1)

    m = 400 # number of examples
    N = int(m/2) # number of points per class
    D = 2 # dimensionality

    X = np.zeros((m,D)) # data matrix where each row is a single example
    y = np.zeros((m,1), dtype='uint8') # labels vector (0 for red, 1 for blue)
    a = 4 # maximum ray of the flower

    for j in range(2):
        ix = range(N*j,N*(j+1))
        t = np.linspace(j*3.12,(j+1)*3.12,N) + np.random.randn(N)*0.2 # theta
        r = a*np.sin(4*t) + np.random.randn(N)*0.2 # radius
        X[ix] = np.c_[r*np.sin(t), r*np.cos(t)]
        y[ix] = j

    X = X.T
    y = y.T

    return X, y

# Load the dataset:
X, y = load_flower_dataset()

# Visualize the dataset:
plt.scatter(X[0, :], X[1, :], c=y, s=40, cmap=plt.cm.Spectral);
```



You will see that you have some red points (which have label $y = 0$) and some blue points (which have label $y = 1$).

More precisely you have created two vectors `X` and `y` (shorthands for X_{train} and y_{train}) containing 400 training examples:

- The numpy-array `X` of dimensions (2, 400), every column of `X` is the feature vector for a single training example and contains the coordinates of the point: $\begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$.
- The numpy-array `y` of dimensions (1,400), every column of `y` contains the label for a single training example (0 for red, 1 for blue).
- The columns of `X` correspond to the columns of `y`, i.e. the point in the first column of `X` corresponds to the first entry in `y` and so on.

Your goal is to build a model to fit this data. In other words, you want the model to define regions of the plane as either red or blue. Red regions will correspond to points (x_0, x_1) where the model returns a value less than 0.5 and with any other parts of the plane being blue regions.

Let's start by building a logistic regression model and then measuring its performance on this problem. You will build it in numpy from scratch.

These are the steps to build your model.

1. Define the structure:

$$\hat{y} = \sigma\left(W \cdot x + b\right) = \sigma\left(\begin{pmatrix} w_0 & w_1 \end{pmatrix} \cdot \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} + b\right).$$

Here the notation is the same as the one in the lecture presentation for this project. In this case, for each example, we take a linear combination $w_0x_0 + w_1x_1 + b$ of the input features x_0 and x_1), where w_0, w_1, b are real numbers, followed by a *sigmoid* activation function $\sigma(\lambda) = \dfrac{1}{1+\exp(-\lambda)}$. The output $\hat{y}$ is the model's predicted probability that the point $x = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$ is blue. Here $W = \begin{pmatrix} w_0 & w_1 \end{pmatrix}$ is known as the weight matrix and b is the bias vector (which in this special case will actually be a single value, a 1×1 matrix).

2. Initialize the model's trainable parameters W and b.

3. The training loop is then as follows:

   - Implement forward propagation: compute an array whose entries are $\hat{y}^{(i)}$ for each training example, with the current values of W and b. Here $\hat{y}^{(i)}$ is the output probability for example in column i of X.
   - Compute the cost J for the current values of the parameters.
   - Implement backward propagation to get the gradients, i.e. the rate of change of the J with respect to each of w_0, w_1 and b at their current values.
   - Update the parameters using these gradients. This is known as gradient descent.

In practice, you'll build helper functions for each of these steps, and then merge them into one function called `model()`. Once you've built `model()` and used it to optimise the parameters, you can make predictions on new data.

Initialize parameters

Complete the function `initialize_parameters` started below. It takes in the dimension of the input layer `n_X` and the dimension of the output layer `n_y` and returns returns a Python dictionary `parameters`. The dictionary `parameters` contains the initialized parameters of the model: the weight matrix `W` and the bias vector `b`.

Note that `n_X = 2` in our example, each point has two input coordinates, and the dimension of the output layer is `n_y = 1`. This means that `W` will be a 1×2 matrix and `b` will be a 1×1 matrix for our example. However we would like our function to work more generally, and be in a form which can be adapted for use later on in this project, so we'd like the user to be able to specify `n_X` and `n_y` as inputs to `initialize_parameters`.

- Use: `np.random.randn(n,m) * 0.01` to randomly initialize a matrix of the required shape for `W`.
- Initialize the bias vector `b` so that all of its entries are zero.
- Return both of these in a dictionary called `parameters` with names *W* and *b* respectively.

Once you have completed the code run this cell.

In [27]:
```
def initialize_parameters(n_X, n_y):
    """
    Argument:
    n_X -- integer
    n_y -- integer

    Returns:
    parameters -- python dictionary containing your parameters:
                    W -- weight matrix
                    b -- bias vector
    """

    #Initialize weight matrix nd bias vector
    W = np.random.randn(n_y,n_X)*0.01 #you need to replace "None" with appropria
    b = np.zeros((n_y,1)) #you need to replace "None" with appropriate code

    #Store W and b in a dictionary parameters
    parameters = {"W": W,
                  "b": b}

    return parameters
```

Forward Propagation

Next you will implement forward propagation for logistic regression:

$$z = WX + b \text{ and then } \hat{y} = \sigma(z)$$

As above, for $\lambda \in \mathbb{R}$, $\sigma(\lambda) = \dfrac{1}{1+\exp(-\lambda)}$ (this is applied entry-wise to z in the above).

Start by implementing the sigmoid function in numpy by replacing *None* in the code cell below with appropriate code. Then run the cell.

In [28]:
```
def sigmoid(z):
    """
    Argument:
    z -- input data, numpy array

    Returns:
    numpy array that contains sigmoid function applied to each
    element of z.
    """
    return (1/(1+np.exp(-z))) #you need to replace "None" with appropriate code
```

Complete the function `forward_propagation` started below. It calculates the output of the logistic regression model, called `y_hat`, and returns (as a tuple) `y_hat` and `z` (in the notation given above).

Once you have completed the code run this cell.

In [29]:
```
def forward_propagation(X, parameters):
    """
    Argument:
    X -- input data, numpy array of dimension (2, 400). Every column of X
```

```
        is the feature vector for a single training example (input of the logis
    parameters -- python dictionary containing the values of W and b to be used
                  to compute the forward propagation.

    Returns:
    y_hat -- numpy array of dimension (1,400). Every column of y_hat is the
             output probability of the logistic regression for that training exa
    z -- numpy array of dimension (1,400), result of linear transformation befor
    """

    # retrieve W and b from the dictionary "parameters"
    W = parameters["W"]
    b = parameters["b"]
    # compute z
    z = np.dot(W, X) + b
    # Compute y_hat
    y_hat = sigmoid(z)
    return y_hat, z
```

Compute cost

Given the predictions $\hat{y}^{(i)}$ on all the training examples, you can compute the cost J as the average over all training examples of the losses:

$$J = \frac{1}{m} \sum_{i=0}^{m} L(\hat{y}^{(i)}, y^{(i)}).$$

Here we are using the binary cross-entropy loss:

$$L(\hat{y}^{(i)}, y^{(i)}) = -\left(y^{(i)} \log\left(\hat{y}^{(i)}\right) + (1 - y^{(i)}) \log\left(1 - \hat{y}^{(i)}\right) \right) .$$

Recall $y^{(i)}$ is the ith component in y, i.e. the label for example i and $\hat{y}^{(i)}$ is the models current prediction of this label, the ith component in $\hat{y}$.

Complete the function `compute_cost()`, started below, to compute the value of the cost J:

1. First compute the vector of losses element-wise using the numpy arrays given as inputs `y_hat` and `y`.
2. Sum the losses and divide by the number of training examples `m`. Call this value `cost`.
3. Cast `cost` as type `float` using `float()` and return it.

Once you have completed the code run this cell.

In [30]:
```
def compute_cost(y_hat, y):
    """
    Arguments:
    y_hat -- numpy array, output of forward propagation
    y -- numpy array of the same shape as y_hat, containing the true labels (0 f

    Returns:
    cost -- a float which is the cross-entropy cost
```

```
    """

    # retrieve number of training examples from the shape of y
    m = y.shape[1]
    # compute the vector of losses by computing the cross-entropy loss element-w
    vector_of_losses = -(y*np.log(y_hat) + (1-y)*np.log(1-y_hat))
    # compute the total cost by averaging the loss over all training examples
    cost = (1/m)*(np.sum(vector_of_losses))
    # cast cost as a float
    cost = float(cost)
    return cost
```

Backpropagation

You can now implement backward propagation. Note that you will complete the code cell below this, much of the code is already given to you but you need to stick with the notation established.

NOTATION: in general for a generic numpy array `M` and a function F with the entries of `M` as inputs, denote by `dM` the numpy array representing the gradient of F with respect to `M`. More precisely `dM` contains the partial derivatives of F with respect to the entries of `M`. For example if

$$M = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} \\ a_{1,0} & a_{1,1} & a_{1,2} \end{pmatrix}$$

and F is a function of $a_{0,0}, a_{0,1}, a_{0,2}, a_{1,0}, a_{1,1}, a_{1,2}$ then

$$dM = \begin{pmatrix} \frac{\partial F}{\partial a_{0,0}} & \frac{\partial F}{\partial a_{0,1}} & \frac{\partial F}{\partial a_{0,2}} \\ \frac{\partial F}{\partial a_{1,0}} & \frac{\partial F}{\partial a_{1,1}} & \frac{\partial F}{\partial a_{1,2}} \end{pmatrix}.$$

Throught this project F will be the cost function J.

Complete the function `backward_propagation()` started below. It takes in a dictionary `parameters` containing the current parameters, `y_hat`, `X` and `y` and it returns a dictionary containing the gradients `dW` and `db` of the cost function with respect to the trainable parameters `W` and `b`.

You are given the key code for calculating `dW` and `db`, it is, in the notation already established.

`dz = y_hat - y dW = np.dot(dz, X.T)/m db = np.sum(dz, axis=1, keepdims=True)/m`

You will see these lines of code in the cell below.

Once you have completed the code run this cell.

In [31]:
```
def backward_propagation(parameters, y_hat, X, y):
    """
    Implement the backward propagation.

    Arguments:
```

```
    parameters -- python dictionary containing W and b to use
    y_hat - from output of forward prop
    X -- input data
    y -- labels vector

    Returns:
    grads -- python dictionary containing the gradients
    """
    # Retrieve the number of training examples from the shape of y
    m = y.shape[1] #you need to replace None here.

    # retrieve W from the dictionary "parameters".
    W = parameters["W"] #you need to replace None here.
    # Backward propagation: calculate dW, and db.
    dz = y_hat - y
    dW = np.dot(dz, X.T)/m
    db = np.sum(dz, axis=1, keepdims=True)/m

    grads = {"dW": dW,
             "db": db}

    return grads
```

Update parameters

Next you will define a function to update parameters using gradient descent. In a similar way to that seen in task A2,

1. W will be updated to

$$W - dW \times \text{learning rate}.$$

and

2. b will be updated to

$$b - db \times \text{learning rate}.$$

Complete the function `update_parameters()` started below. It takes in the dictionary `parameters` containing the current parameters `W` and `b`, the dictionary `grads` containing the current gradient `dW`, `dB` and the learning rate (which defaults to the value 1.2). **Once you have completed the code run this cell.**

In [32]:
```
def update_parameters(parameters, grads, learning_rate = 1.2):
    """
    Arguments:
    parameters -- python dictionary containing the parameters
    grads -- python dictionary containing the gradients

    Returns:
    parameters -- python dictionary containing the updated parameters
    """
    # Retrieve a copy of each parameter from the dictionary "parameters". Use co

    W = copy.deepcopy(parameters["W"])
    b = copy.deepcopy(parameters["b"])
```

```
    # Retrieve each gradient from the dictionary "grads"
    dW = grads["dW"]
    db = grads["db"]

    # Update each parameter
    W = W - learning_rate * dW
    b = b - learning_rate * db
    # Store the new parameters in the dictionary parameters
    parameters = {"W": W, "b": b}
    return parameters
```

Model

Now you will put everything together into a function `model` where you will apply `update_parameters` iteratively to update the parameters a number of times.

Complete the function `model` started below. It takes as inputs the training data `X`, `y`, a positive integer `num_iterations` which is the number of times the parameters will be updated (defaulting to $10,000$) and a boolean `print_cost` which, when set to `True`, will mean the function prints out some costs as the values of the parameters are updated). `model` return a dictionary containing the optimised parameters at the end of this process. **Once you have completed the code run this cell.**

In [33]:
```
def model(X, y, num_iterations = 10000, print_cost=False):
    """
    Arguments:
    X -- numpy array, input data
    y -- numpy array, true labels
    num_iterations -- number of iterations in the gradient descent loop
    print_cost -- if True, print the cost every 1000 iterations

    Returns:
    parameters -- python dictionary which contains the optimized parameters
                  learnt by the model.
    """
    # Optional: to control the random seed of the initialization uncomment next
    # np.random.seed(3)

    # retrieve n_x and n_y from X and y
    n_x = X.shape[0]
    n_y = y.shape[0]
    # Initialize parameters
    parameters = initialize_parameters(n_x, n_y)
    # Training loop (gradient descent)
    for i in range(0, num_iterations):

        # Forward propagation
        y_hat,z = forward_propagation(X, parameters)
        # Compute the cost
        cost = compute_cost(y_hat, y)
        # Backpropagation
        backward = backward_propagation(parameters, y_hat, X,y)
        # Update parameters (use learning_rate = 1.2)
        parameters = update_parameters(parameters, backward, learning_rate = 1.2
        # Print the cost every 1000 iterations
        if print_cost and i % 1000 == 0:
            print ("Cost after iteration %i: %f" %(i, cost))
```

```
    learned_parameters = parameters

    # return learned parameters
    return learned_parameters
```

Predict on new data

Below you are given a complete function `predict` that uses the learned parameters (output of `model`) and the `forward_propagation` function to predict a class for each example in the input matrix `X`. **Run the code cell below.**

In [34]:
```
def predict(learned_parameters, X):
    """
    Arguments:
    parameters -- python dictionary containing the learned parameters given by m
    X -- numpy array input data

    Returns
    predictions -- vector of predictions for each column of X (red: 0 / blue: 1)
    """

    # Compute predicted probabilities using the learned parameters

    y_hat, z = forward_propagation(X, learned_parameters)

    # Classify as 0 or 1 using 0.5 as a threshold

    predictions = (y_hat > 0.5)

    return predictions
```

Plot the results and calculate accuracy

You are given the complete function `plot_decision_boundary` below which plots the decision boundary of the trained model along with the original data. It takes as arguments the `learned_parameters` (as output by the `model` function), the model, the training examples `X` and the true labels `y`. **Run the code cell below.**

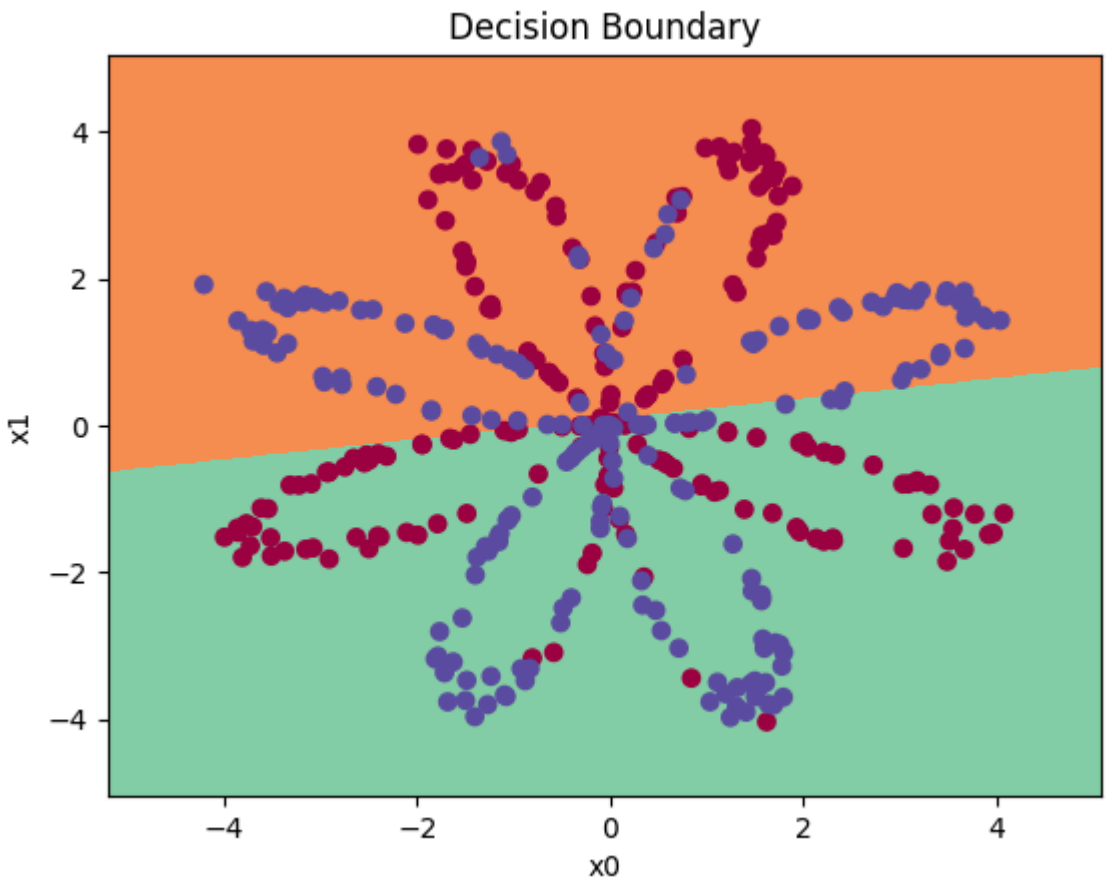
In [35]:
```
# Function to plot decision boundary

def plot_decision_boundary(learned_parameters, X, y):

    # Set min and max values and give it some padding
    x_min, x_max = X[0, :].min() - 1, X[0, :].max() + 1
    y_min, y_max = X[1, :].min() - 1, X[1, :].max() + 1
    h = 0.01

    # Generate a grid of points with distance h between them
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

    # Predict the function value for the whole grid
    Z = predict(learned_parameters,np.c_[xx.ravel(), yy.ravel()].T)
    Z = Z.reshape(xx.shape)

    # Plot the contour and training examples
```

```
    plt.contourf(xx, yy, Z, cmap=plt.cm.Spectral)
    plt.ylabel('x1')
    plt.xlabel('x0')
    plt.scatter(X[0, :], X[1, :], c=y, cmap=plt.cm.Spectral)
    plt.title("Decision Boundary")
    plt.show()
```

Now you can train your model using the training examples, and plot the decision boundary for the learned parameters, alongside the orginal data. **Run the code cell below**.

In [36]:
```
# Train your model
learned_parameters = model(X, y, num_iterations = 10000, print_cost=True)

# Plot the decision boundary
plot_decision_boundary(learned_parameters, X, y)
```

```
Cost after iteration 0: 0.694565
Cost after iteration 1000: 0.673146
Cost after iteration 2000: 0.673146
Cost after iteration 3000: 0.673146
Cost after iteration 4000: 0.673146
Cost after iteration 5000: 0.673146
Cost after iteration 6000: 0.673146
Cost after iteration 7000: 0.673146
Cost after iteration 8000: 0.673146
Cost after iteration 9000: 0.673146
```



As you should see logistic regression doesn't do a good job in predicting the labels for this dataset. This is because the dataset is not linearly separable, and therefore a simple logistic regression performs poorly on this data.

Complete the code cell below with a formula for the accuracy of the model. This should be the precentage of points for which the model gives the correct classification. **Then run the cell.**

In [37]:
```
# Print accuracy

predictions = predict(learned_parameters, X)
accuracy = np.mean(predictions == y)*100 # replace 'None' with code which calcul
            # which the prediction agrees with the actual label.

print ('Accuracy of logistic regression: %d ' % accuracy +
       '% ' + "(percentage of correctly labelled datapoints)")
```

```
Accuracy of logistic regression: 47 % (percentage of correctly labelled datapoint
s)
```

A4 - Build a hidden layer Neural Network (worth approximately 20% of the marks)

You are now going to build a full Neural Network with one hidden layer. This will allow to model to learn more complex decision boundaries and not just linear ones. This will result in an improved model compared to that you obtained in task A3.

In practice you are going to repeat all the steps of task A3 but this time with an extra layer to the network. Set the hidden layer to have 4 neurons (in the code this dimension is denoted `n_h`). You may find it helpful to refer to the project lecture when you are completing this section.

The forward propagation has the form `linear` -> `tanh` -> `linear` -> `sigmoid`. So this will take the form below where W and T are appropriately sized matrices containing weights and b and c are appropriately sized column vectors for the biases (some of the dimensions of these objects will now depend on `n_h`). The input matrix X is the same as in task A3, z, a, v give intermediate output with $\hat{y}$ the final output.

$$z = W \cdot X + b$$
$$a = \tanh(z)$$
$$v = T \cdot a + c$$
$$\hat{y} = \sigma(v)$$

The code for the new backward propogation function, in this notation, is as follows and you can cut and paste this into your code as appropriate.

`dv = y_hat - y dT = np.dot(dv, a.T)/m dc = np.sum(dv, axis=1, keepdims=True)/m dz = np.dot(T.T, dv) * (1 - np.power(a, 2)) dW = np.dot(dz, X.T)/m db = np.sum(dz, axis=1, keepdims=True)/m`

The cost is the same as for logistic regression, so there is no need to implement it again.

Neural Network Model

Below gives a basic structure for your final model, you need to build all the helper functions from scratch this time.

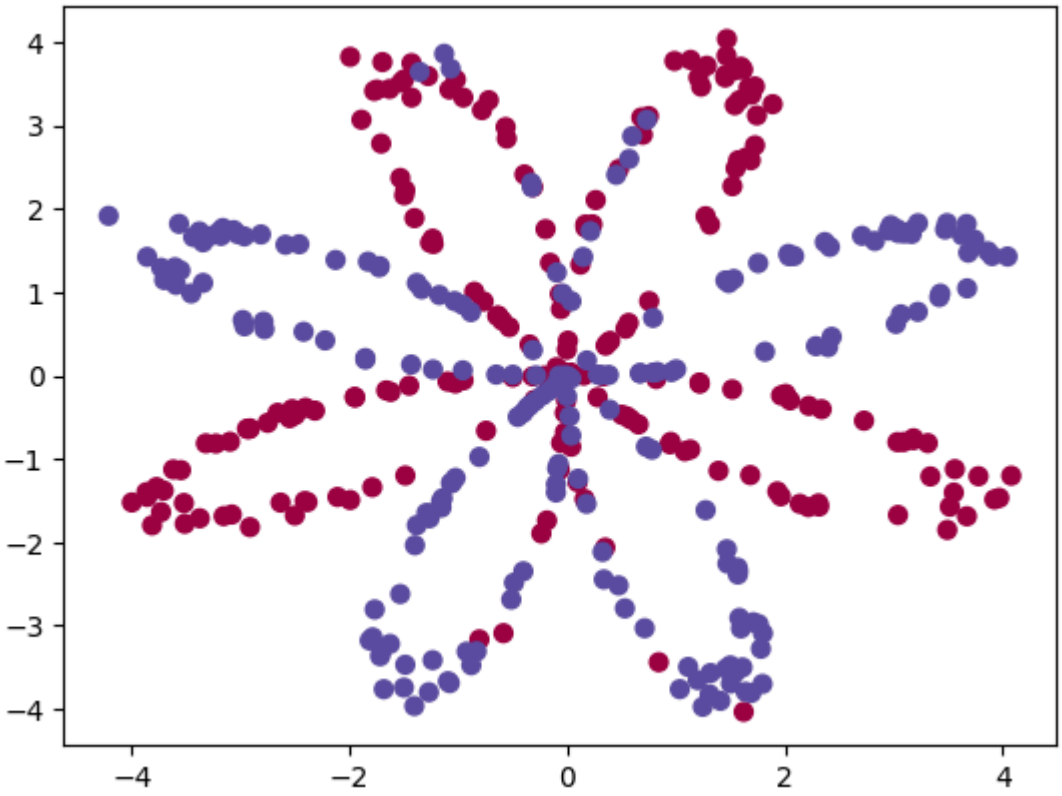
Specifically, using an approach similar to that seen in A3, you should

- Complete the function below `model_nn`, including all the necessary helper functions.
- Include a function `predict` which takes in the dictionary of paremeters and input data 'X' and returns predictions of the labels for each example.
- Produce a visualisation of the regions of the plane predicted to be red/blue by your model and the original data.
- Calculate and print the accuracy of the model (the percentage of points for which the model predicts the correct label).
- Make some comments, in a markdown cell, which compare the model developed in A3 to the model developed in A4.

You may use code from Task A3 here but be careful to rename functions and variables as appropriate to avoid clashes with any values established for this notebook in Task A3.

Edit the code cell below and insert further code and markdown cells in which to answer this task

In [38]:
```
X, y = load_flower_dataset()
plt.scatter(X[0, :], X[1, :], c=y, s=40, cmap=plt.cm.Spectral);
```



In [39]:
```
def model_nn(X, y, n_h = 4, num_iterations = 10000, print_cost=False):

    # Optional: to control the random seed of the initialization uncomment next
    # np.random.seed(3)

    # retrieve n_x and n_y
```

```
    n_x = X.shape[0]
    n_y = y.shape[0]

    # Initialize parameters
    parameters = initialize_parameters4(n_x, n_y,n_h)

    # Training loop
    for i in range(0, num_iterations):

        # Forward propagation
        y_hat, v, a= forward_propagation4(X, parameters)

        # Compute cost
        cost = compute_cost4(y_hat, y)

        # Backpropagation
        grads = backward_propagation4(parameters, y_hat, X, y, a)

        # Update parameters
        parameters = update_parameters4(parameters, grads, learning_rate = 1.2)

        # Print the cost every 1000 iterations
        if print_cost and i % 1000 == 0:
            print ("Cost after iteration %i: %f" %(i, cost))

        learned_parameters = parameters

    return learned_parameters
```

In [40]:
```
def initialize_parameters4(n_x, n_y, n_h):
    """
    Argument:
    n_X -- integer
    n_y -- integer

    Returns:
    parameters -- python dictionary containing your parameters:
                    W -- weight matrix
                    b -- bias vector
    """

    #Initialize weight matrix nd bias vector
    W = np.random.randn(n_h,n_x) * 0.01
    b = np.zeros(n_y)

    T = np.random.randn(1,n_h)
    c = np.zeros(n_y)

    #Store W and b in a dictionary parameters
    parameters = {"W": W,
                  "b": b,
                  "T": T,
                  "c": c}

    return parameters
```

In [41]:
```
def forward_propagation4(X, parameters):
```

```
    # retrieve W and b from the dictionary "parameters"
    W = parameters["W"]
    b = parameters["b"]
    T = parameters["T"]
    c = parameters["c"]

    # compute z
    z = np.dot(W,X) + b

    #tanh activation function
    a = np.tanh(z)
    v = np.dot(T,a) + c

    # Compute y_hat
    y_hat = sigmoid(v)

    return y_hat, v, a
```

In [42]:
```
def compute_cost4(y_hat, y):
    """
    Arguments:
    y_hat -- numpy array, output of forward propagation
    y -- numpy array of the same shape as y_hat, containing the true labels (0 f

    Returns:
    cost -- a float which is the cross-entropy cost
    """

    # retrieve number of training examples from the shape of y
    m = y.shape[1]

    # compute the vector of losses by computing the cross-entropy loss element-w
    cost = -(1/m) * np.sum(y*np.log(y_hat) + (1 - y)* np.log(1 - y_hat))

    # compute the total cost by averaging the loss over all training examples

    # cast cost as a float
    float(cost)

    return cost
```

In [43]:
```
def backward_propagation4(parameters, y_hat, X, y, a):

    # Retrieve the number of training examples from the shape of y
    m = y.shape[1]

    # retrieve W and T from the dictionary "parameters".
    W = parameters["W"]
    T = parameters["T"]

    # Backward propagation: calculate dW, db, dc, dT, dz.
    dv = y_hat - y
    dT = np.dot(dv, a.T)/m
    dc = np.sum(dv, axis=1, keepdims=True)/m
    dz = np.dot(T.T, dv) * (1 - np.power(a, 2))
    dW = np.dot(dz, X.T)/m
    db = np.sum(dz, axis=1, keepdims=True)/m

    grads = {"dW": dW,
```

```
                "db": db,
                "dT": dT,
                "dc": dc}

    return grads
```

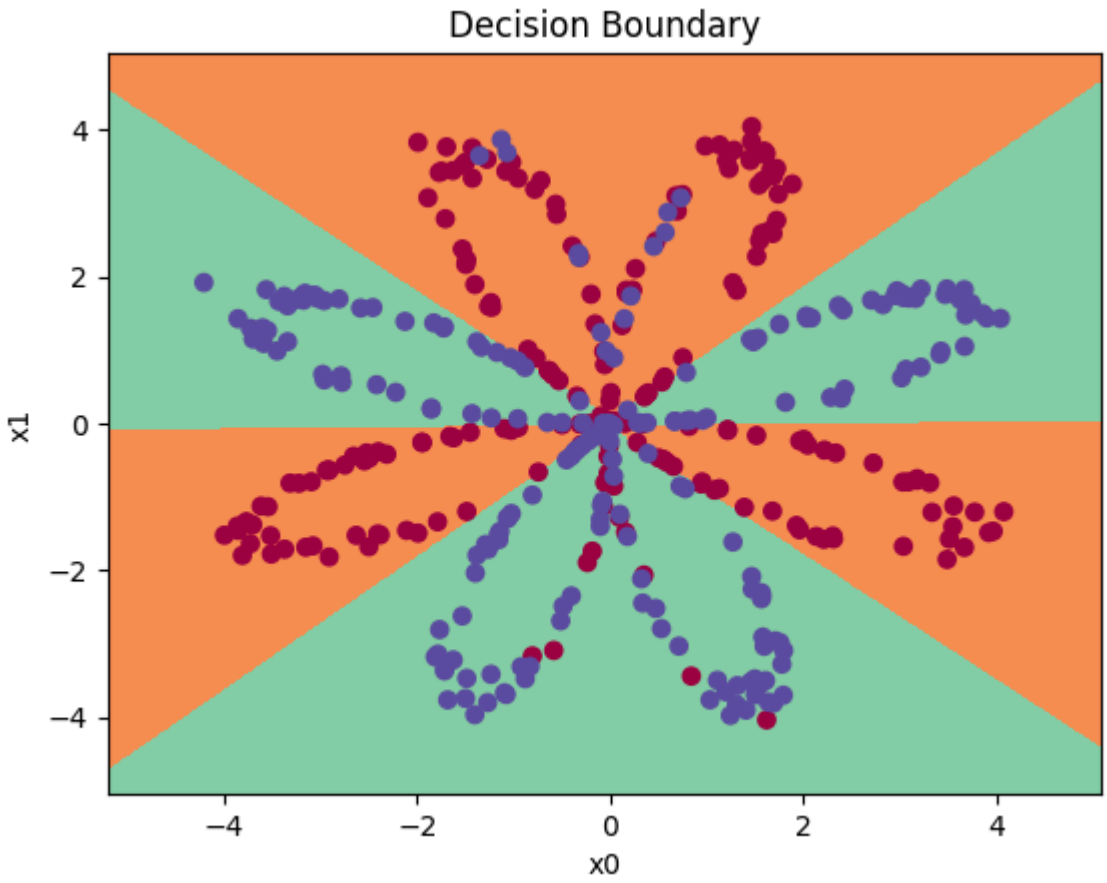
In [44]:
```
def update_parameters4(parameters, grads, learning_rate = 1.2):

    # Retrieve a copy of each parameter from the dictionary "parameters". Use co

    W = copy.deepcopy(parameters["W"])
    b = parameters["b"]
    T = copy.deepcopy(parameters["T"])
    c = parameters["c"]

    # Retrieve each gradient from the dictionary "grads"
    dW = grads["dW"]
    db = grads["db"]
    dT = grads["dT"]
    dc = grads["dc"]

    # Update each parameter
    W = W - dW * learning_rate

    b = b - db * learning_rate

    T = T - dT * learning_rate

    c = c - dc * learning_rate

    # Store the new parameters in the dictionary parameters
    parameters = {"W":W,
                  "b":b,
                  "T":T,
                  "c":c}

    return parameters
```

In [45]:
```
def predict4(learned_parameters, X):
    """
    Arguments:
    parameters -- python dictionary containing the learned parameters given by m
    X -- numpy array input data

    Returns
    predictions -- vector of predictions for each column of X (red: 0 / blue: 1)
    """

    # Compute predicted probabilities using the learned parameters

    y_hat, v,a = forward_propagation4(X, learned_parameters)

    # Classify as 0 or 1 using 0.5 as a threshold

    predictions = (y_hat > 0.5)

    return predictions
```

In [46]:
```
def plot_decision_boundary4(learned_parameters, X, y):

    # Set min and max values and give it some padding
    x_min, x_max = X[0, :].min() - 1, X[0, :].max() + 1
    y_min, y_max = X[1, :].min() - 1, X[1, :].max() + 1
    h = 0.01

    # Generate a grid of points with distance h between them
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

    # Predict the function value for the whole grid
    Z = predict4(learned_parameters,np.c_[xx.ravel(), yy.ravel()].T)
    Z = Z.reshape(xx.shape)

    # Plot the contour and training examples
    plt.contourf(xx, yy, Z, cmap=plt.cm.Spectral)
    plt.ylabel('x1')
    plt.xlabel('x0')
    plt.scatter(X[0, :], X[1, :], c=y, cmap=plt.cm.Spectral)
    plt.title("Decision Boundary")
    plt.show()
```

In [47]:
```
# Train your model
learned_parameters4 = model_nn(X, y, num_iterations = 10000, print_cost=True)

# Plot the decision boundary
plot_decision_boundary4(learned_parameters4, X, y)
```

```
Cost after iteration 0: 0.689940
Cost after iteration 1000: 0.281784
Cost after iteration 2000: 0.269405
Cost after iteration 3000: 0.262034
Cost after iteration 4000: 0.238920
Cost after iteration 5000: 0.225722
Cost after iteration 6000: 0.221466
Cost after iteration 7000: 0.218798
Cost after iteration 8000: 0.216799
Cost after iteration 9000: 0.215189
```



In [48]:
```
# Print accuracy

predictions4 = predict4(learned_parameters4, X)

accuracy = np.mean(predictions4 == y)*100

print ('Accuracy: %d ' % accuracy +
       '% ' + "(percentage of correctly labelled datapoints)")
```

```
Accuracy: 90 % (percentage of correctly labelled datapoints)
```

Section B (worth approximately 40%)

Remember you are free to use sklearn in this section.

Task B1

- How can we further improve neural networks? You may wish to talk about underfitting, overfitting, regularisation and use examples to illustrate you ideas.

Task B2

- What are some other machine learning models (e.g. decision trees). Give some examples. What are improvements to these (e.g random forest, xgboost in the case of decision trees)?

You may wish to use datasets from https://archive.ics.uci.edu/ in this section.

Semi-supervised Random Forest

A supervised machine learning model is a model in which all of the datapoints are labelled with the statistic of interest, while an unsupervised model is one where none of the datapoints are labelled with the statistic of interest. Unsupervised models are generally easier to create, since it avoids the need to label the datapoints, which can be hard or impossible in real-life scenarios. However, unsupervised models tend to be less accurate as a result, since it can't verify and optimise its outputs. A semi-supervised model combines the best parts of both, as only some of the data points that the model analyses are labelled. This means that the set of data is easier to obtain, since less labelling must be done, but the model is still able to verify its results and therefore become more accurate.

We will be using a semi-supervised random forest method to attempt to classify whether websites are phishing sites based on the website's URL. This uses the 'PhiUSIIL Phishing URL (Website)' dataset from UC Irvine Machine Learning Repository. [2]

All of the data is labelled in this case, so to simulate a semi-supervised model, we will be manually 'unlabelling' a random selection of 80% of the labelled data.

A random forest is made up of some number of decision trees. Each individual decision tree is given a bootstrap of the training data, then attempts to split this bootstrap into smaller subsets based on specific conditions. For example, such a condition could be whether the length of the URL is less than 50 characters. It settles on the condition that best separated phishing websites from legitimate websites. The tree then repeats this process for the newly-created subsets, until all of its subsets contain only phishing websites or only legitimate websites (or at least as close as possible). For a specific website that the model wishes to classify, each decision tree will make a prediction about the website based on the set of conditions that it created from the training data. It does this by assigning that website to one of its created subsets, by following the created set of conditions.

In the case of a semi-supervised version, the model begins by making a random forest based on the labelled training data. It then uses this random forest to attempt to label each website in the unlabelled training data. If a specified level of confidence is achieved, the model will label that website with the corresponding classification, then add it to the labelled training data. This process is then repeated using the new set of labelled training data, again attempting to label the unlabelled training data. This process repeats until no more of the websites in the unlabelled training data are able to be labelled by the random forest.

Finally, we can use this developed random forest to attempt to classify the websites in the testing data.

In [49]:
```
import pandas as pd # Note, some of these libraries have already been imported i
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.semi_supervised import SelfTrainingClassifier
from sklearn.metrics import classification_report

df = pd.read_csv("PhiUSIIL_Phishing_URL_Dataset.csv") # Reading data from data s
df_small = df.sample(n=50000, random_state=16) # We take a random sample of the
# would take a lot of time and computing power to go through
```

In [50]:
```
def analyse_dataset(selected_features, N): # we define the creation of the rando
    X = df_small[selected_features] # we choose to make a random forest using on
    Y = df_small["label"]

    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, ran
    # using random_state to ensure reproducibility

    rng = np.random.RandomState(16) # set of random numbers
    mask = rng.rand(len(Y_train)) < 0.8 # 'unlabelling' 80% of training data
    Y_train[mask] = -1  #tells SKlearn that the masked values are unlabelled

    base_rf = RandomForestClassifier(n_estimators=N, random_state=16) # Creates
    self_training_model = SelfTrainingClassifier(base_rf, threshold=0.75) # Defi

    self_training_model.fit(X_train, Y_train) # Growing random forest on X_train
    Y_pred = self_training_model.predict(X_test)
    print(classification_report(Y_test, Y_pred)) # Returns information about the

# Note that scaling is NOT used here because each decision tree splits the data
# actual magnitude is irrelevant
```

In the below analysis, the row labelled 0 refers to legitimate websites, while the row labelled 1 refers to phishing websites. The precision for each row is the proportion of the model's assignment of that label that was correct. The recall is the proportion of the websites of that type that the model labelled correctly. The f1-score accounts for both the precision and the recall by taking a harmonic mean. The macro average is the mean of the results from the phishing websites and the legitimate websites. The weighted average is calculated by accounting for the sizes of each [5]

In [51]:
```
selected_features = ["URLLength"]
analyse_dataset(selected_features,100)
# The model only analyses the websites based on URL length. 100 decision trees a
```

```
              precision    recall  f1-score   support

           0       0.88      0.50      0.64      4427
           1       0.71      0.94      0.81      5573

    accuracy                           0.75     10000
   macro avg       0.79      0.72      0.72     10000
weighted avg       0.78      0.75      0.73     10000
```

In [52]:
```
selected_features = ["URLLength"]
analyse_dataset(selected_features,1000)
# The model only analyses the websites based on URL length. 1000 decision trees
```

```
              precision    recall  f1-score   support

           0       0.88      0.50      0.64      4427
           1       0.71      0.94      0.81      5573

    accuracy                           0.75     10000
   macro avg       0.79      0.72      0.72     10000
weighted avg       0.78      0.75      0.73     10000
```

In [53]:
```
selected_features = ["URLLength"]
analyse_dataset(selected_features,1)
# The model only analyses the websites based on URL length. 1 decision trees is
```

```
              precision    recall  f1-score   support

           0       0.89      0.48      0.63      4427
           1       0.70      0.95      0.81      5573

    accuracy                           0.75     10000
   macro avg       0.80      0.72      0.72     10000
weighted avg       0.78      0.75      0.73     10000
```

We see that the number of decision trees used by the model has almost no effect on the model's accuracy. However, using multiple decision trees instead of just 1 massively reduces the effects of random chance on the model. With only one tree, there is a significant chance that the tree, and therefore the whole model, develops certain biases and errors. For example, if, in the bootstrap given to that tree, all of the websites with URLs that are 25 characters long happen to be phishing websites, then the whole model will automatically label websites with URLs that are 25 characters long as phishing websites, when this is probably not accurate. Using multiple decision trees almost entirely removes the risk of biases like this developing, since each tree is given a unique bootstrap of the data.

In [54]:
```
selected_features = ["URLLength","SpacialCharRatioInURL"] # Note that 'special'
analyse_dataset(selected_features,100)
# The model only analyses the websites based on URL length and the ratio of spec
```

```
              precision    recall  f1-score   support

           0       0.90      0.64      0.74      4427
           1       0.76      0.94      0.84      5573

    accuracy                           0.81     10000
   macro avg       0.83      0.79      0.79     10000
weighted avg       0.82      0.81      0.80     10000
```

Note that there is only a marginal improvement in the model's accuracy, implying that the ratio of special characters in the URL is not a very good indicator of whether a website is a phishing site

In [55]:
```
selected_features = ["URLLength","LetterRatioInURL"]
analyse_dataset(selected_features,100)
# The model only analyses the websites based on URL length and the ratio of lett
```

```
              precision    recall  f1-score   support

           0       0.96      0.82      0.88      4427
           1       0.87      0.97      0.92      5573

    accuracy                           0.90     10000
   macro avg       0.91      0.90      0.90     10000
weighted avg       0.91      0.90      0.90     10000
```

Replacing the ratio of special characters in the URL with the ratio of letters in the URL leads to a massive increase in the model's accuracy, which implies that the ratio of letters in the URL is a significantly more important metric than the proportion of special characters in the URL

In [56]:
```
selected_features = ["URLLength","LetterRatioInURL","IsHTTPS"]
analyse_dataset(selected_features,100)
# The model analyses the websites based on URL length, the ratio of letters in t
# 100 decisions tree are used
```

```
              precision    recall  f1-score   support

           0       0.98      0.94      0.96      4427
           1       0.95      0.99      0.97      5573

    accuracy                           0.96     10000
   macro avg       0.97      0.96      0.96     10000
weighted avg       0.97      0.96      0.96     10000
```

In [57]:
```
selected_features = ["URLLength","LetterRatioInURL","IsHTTPS","DegitRatioInURL",
analyse_dataset(selected_features,100)
# The model analyses the websites based on URL length, the ratio of letters in t
# the ratio of digits in the URL, and the ratio of special characters in the URL
# 100 decisions tree are used
```

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      4427
           1       0.99      0.99      0.99      5573

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

After giving the model only 5 features to analyse, the model achieves 99% accuracy, which shows that just these 5 features are heavily predictive of whether a website is a phishing website. This suggests that the model is very effective. Overall, we can see that the model's precision for legitimate sites is always higher than for phishing sites, whereas the recall for legitimate sites is always lower than for phishing sites. This implies that the model frequently mislabels legitimate websites as phishing websites, and rarely mislabels phishing websites as legitimate.

K means clustering model

Overview of the model

K-means clustering is an unsupervised machine learning model which separates a collection of data points into 'clusters' of points with similar features. In this model we will use a Mall Customer dataset which consists of a customer database.

Variables used :

- Age
- Spending score
- Annual income

How the K means clustering process works

First, we start off by deciding how many clusters we want to separate our data into, our k-value.

We then randomly choose k-points on our graph to be centroids. For each datapoint we calculate the Euclidean distance to each of the centroids. Each data point is assigned to the centroid of which it is closest to – forming clusters.

Then using our newly formed clusters, we calculate the average of all the datapoints within each cluster – giving us our new centroids. Repeat the process of calculating the distance to each centroid and assigning data points to centroids – this means some data points will be reassigned to new clusters. Stop when the centroids & the clusters no longer change.

The dataset is now separated into k-clusters.

Idea of optimal k value

The model will explore the idea of finding an optimal k value of our model ie. to seperate our data into groups which provide us with the most useful amount of information.

In [58]:
```
import os
os.environ["OMP_NUM_THREADS"] = "1" # We have done this to stop a UserWarning we

from sklearn.cluster import KMeans #import methods from sklearn
from sklearn.preprocessing import MinMaxScaler # Allows for scaling of the data

#Read the MallCustomers.csv file to DataFrame MallCustomers.
MallCustomers = pd.read_csv("MallCustomers.csv")
```

Requirement for scaling

Here we use MinMaxScaler to ensure all variable have an equal impact on the distance calculation as the values for spending score and annual income tend to be higher than age. [7]

Elbow method

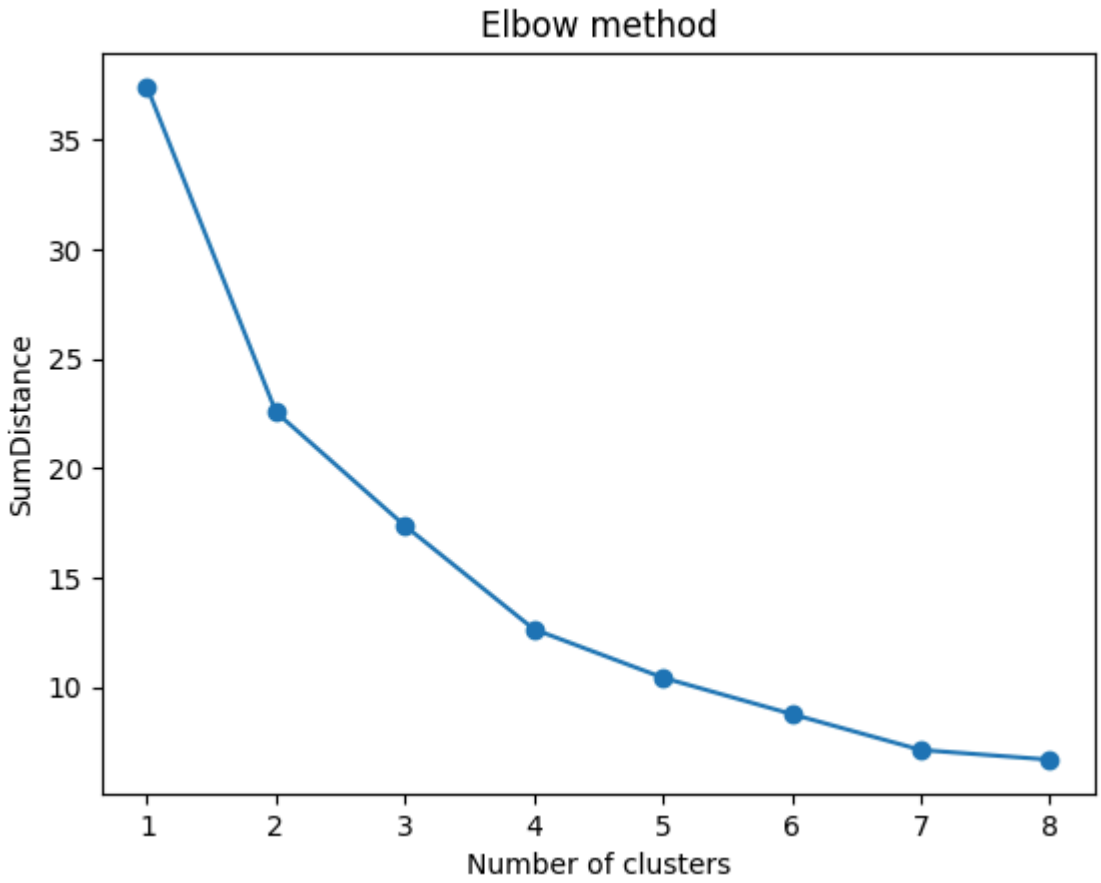
To determine our optimal k-value we use the elbow method.

If we have too few clusters we can end up with underfitting where the model is unable to capture the patterns in the data. Now, if we increase the number of clusters until the point where every point is its own cluster, we end up with overfitting where the model learns the data too closely.

To find an optimal k-value we use a Within Cluster Sum of Squares (WCSS) calculation. This calculates the total sum of each point to its centroid. We want to minimise this.

We can plot a graph of the WCSS calculations for a number of k values, featured below. Around k = 5 is ideal as our WCSS calculation doesn't decrease much now if we continue to increase k, meaning, we make the grouping process more complicated for a very small return in accuracy.

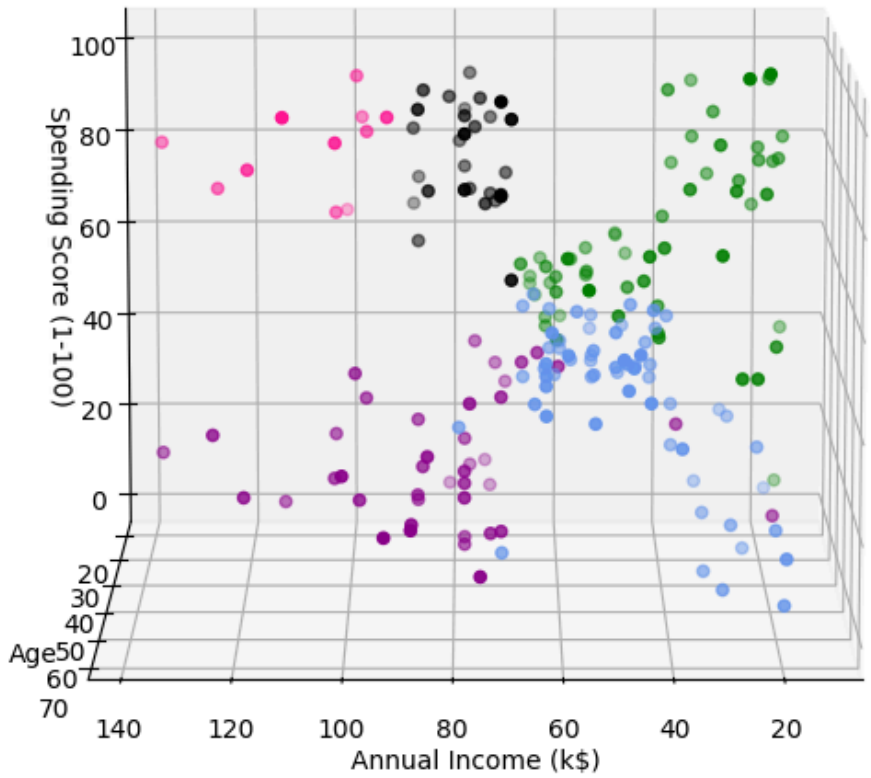
In [59]:
```
X = MallCustomers[['Annual Income (k$)', 'Age', 'Spending Score (1-100)']] #igno
scaler = MinMaxScaler() #creating scaler object
scaler.fit(X) #finds max and min
Xscaled = scaler.transform(X) #actually scaling the objects

SumDistance = []
len(MallCustomers)
for i in range(1,9): #Run k-means clustering process for k = 1...8
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(Xscaled)
    SumDistance.append(kmeans.inertia_) #gives the sum of squared distances from

plt.plot(range(1,9), SumDistance, marker='o') #plotting the elbow method
plt.title('Elbow method')
plt.xlabel('Number of clusters')
plt.ylabel('SumDistance')
plt.show()
```

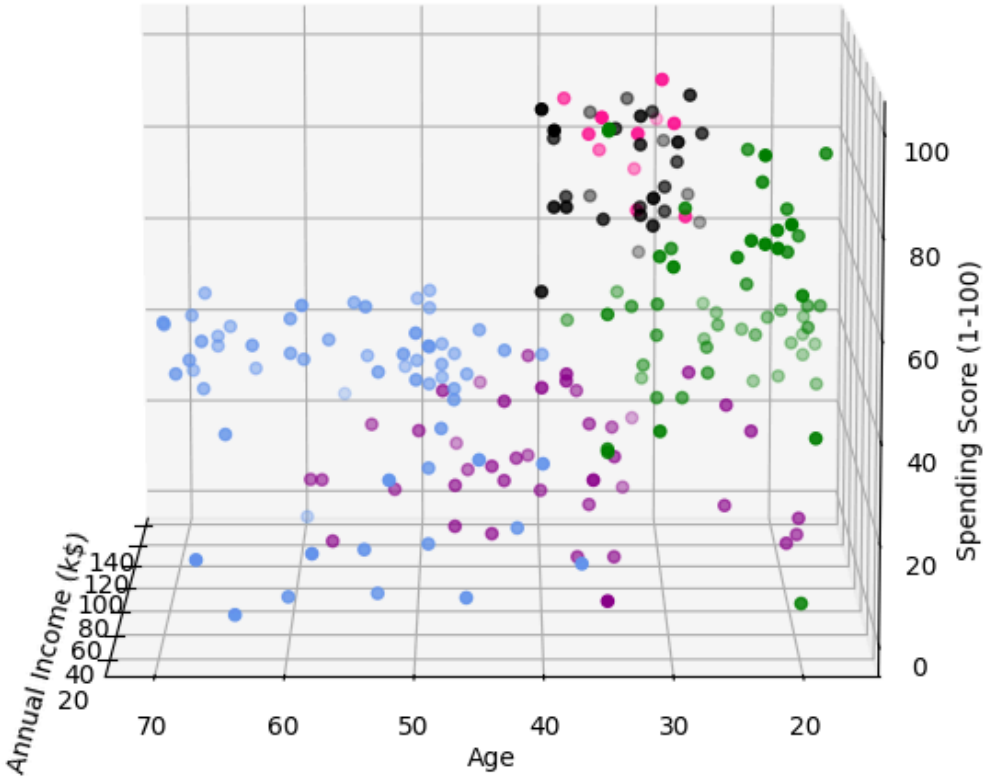


From the graph we can see an optimal k value of 5 or 6, but we will choose 5 for a slightly less crowded plot

In [60]:
```
km = KMeans(n_clusters = 5) #Choosing our k value from the elbow method (number
km.fit(Xscaled) #works out best clusters based on features provided, by the meth


labels=km.labels_ #sorts into 5 clusters
MallCustomers['ClusterVals']=labels #include cluster value in data set

Cluster0 = MallCustomers[MallCustomers['ClusterVals']==0] #Creating a dataframe
Cluster1 = MallCustomers[MallCustomers['ClusterVals']==1]
Cluster2 = MallCustomers[MallCustomers['ClusterVals']==2]
Cluster3 = MallCustomers[MallCustomers['ClusterVals']==3]
Cluster4 = MallCustomers[MallCustomers['ClusterVals']==4]

#From one angle
fig = plt.figure(figsize=(8,10))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(Cluster0['Annual Income (k$)'],Cluster0['Age'],Cluster0['Spending Sco
ax.scatter(Cluster1['Annual Income (k$)'],Cluster1['Age'],Cluster1['Spending Sco
ax.scatter(Cluster2['Annual Income (k$)'],Cluster2['Age'],Cluster2['Spending Sco
ax.scatter(Cluster3['Annual Income (k$)'],Cluster3['Age'],Cluster3['Spending Sco
ax.scatter(Cluster4['Annual Income (k$)'],Cluster4['Age'],Cluster4['Spending Sco
ax.set_xlabel('Annual Income (k$)')
ax.set_ylabel('Age')
ax.set_zlabel('Spending Score (1-100)')
ax.view_init(azim=90,elev=10)


#From another angle
```

```
fig = plt.figure(figsize=(8,10))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(Cluster0['Annual Income (k$)'],Cluster0['Age'],Cluster0['Spending Sco
ax.scatter(Cluster1['Annual Income (k$)'],Cluster1['Age'],Cluster1['Spending Sco
ax.scatter(Cluster2['Annual Income (k$)'],Cluster2['Age'],Cluster2['Spending Sco
ax.scatter(Cluster3['Annual Income (k$)'],Cluster3['Age'],Cluster3['Spending Sco
ax.scatter(Cluster4['Annual Income (k$)'],Cluster4['Age'],Cluster4['Spending Sco
ax.set_xlabel('Annual Income (k$)')
ax.set_ylabel('Age')
ax.set_zlabel('Spending Score (1-100)')
ax.view_init(azim=180,elev=10)
```





Graph plotting

Here we have plotted our dataset (from 2 different angles) after being seperated into 5 clusters with similar features eg. similar purchasing habits or similar demographics.

This can then be used in targeted marketing.

How we can now use the model

Examples of what each cluster could represent :

- Blue medium-low income, middle-aged to old, medium spending

  > An okay group.

- Black medium income, young, high income, medium-low spending

  > An okay group.

- Green low income, young, high spending

  > An ideal group.

- Pink high income, young, high income, high spending

  > An ideal group.

- Purple high income, wide range of ages, low spending

  > Not an ideal group for marketing.

Comparison of Models

In this part of the project, we aim to compare different models on the Wisconsin data set for breast cancer cells [1]

Importing all the necessary libraries:

In [61]:
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import ConfusionMatrixDisplay
```

Loading the data:

In [62]:
```
data = load_breast_cancer()

df = pd.DataFrame(data.data, columns=data.feature_names)

#0 is malignant (cancer), 1 is benign (not cancer)
df["target"] = data.target
```

Printing basic information about the dataset

In [63]:
```
print(f"Number of samples: {df.shape[0]}")
print(f"Number of features: {df.shape[1] - 1}")  #-1 for target
print(f"\nTarget classes:")
print(f"0 = {data.target_names[0]} (cancerous)")
print(f"1 = {data.target_names[1]} (not cancerous)")

print(f"\nClass counts:")
print(f"Malignant: {(df['target'] == 0).sum()}")
print(f"Benign: {(df['target'] == 1).sum()}")

#Checking for any missing data (Since the data set is already cleaned for us, we
print(f"\nMissing values: {df.isnull().sum().sum()}")
```

```
Number of samples: 569
Number of features: 30

Target classes:
0 = malignant (cancerous)
1 = benign (not cancerous)

Class counts:
Malignant: 212
Benign: 357

Missing values: 0
```

Exploring the data:

We will focus on 6 key features that are easiest to understand. These describe the average measurements of the tumor cells. We also print summary statistics for these features.

In [64]:
```
key_features = [
    "mean radius",
    "mean texture",
    "mean perimeter",
    "mean area",
    "mean smoothness",
    "mean compactness",
]

print("\nSummary statistics for key features:")
print(df[key_features].describe().round(3).to_string())
```

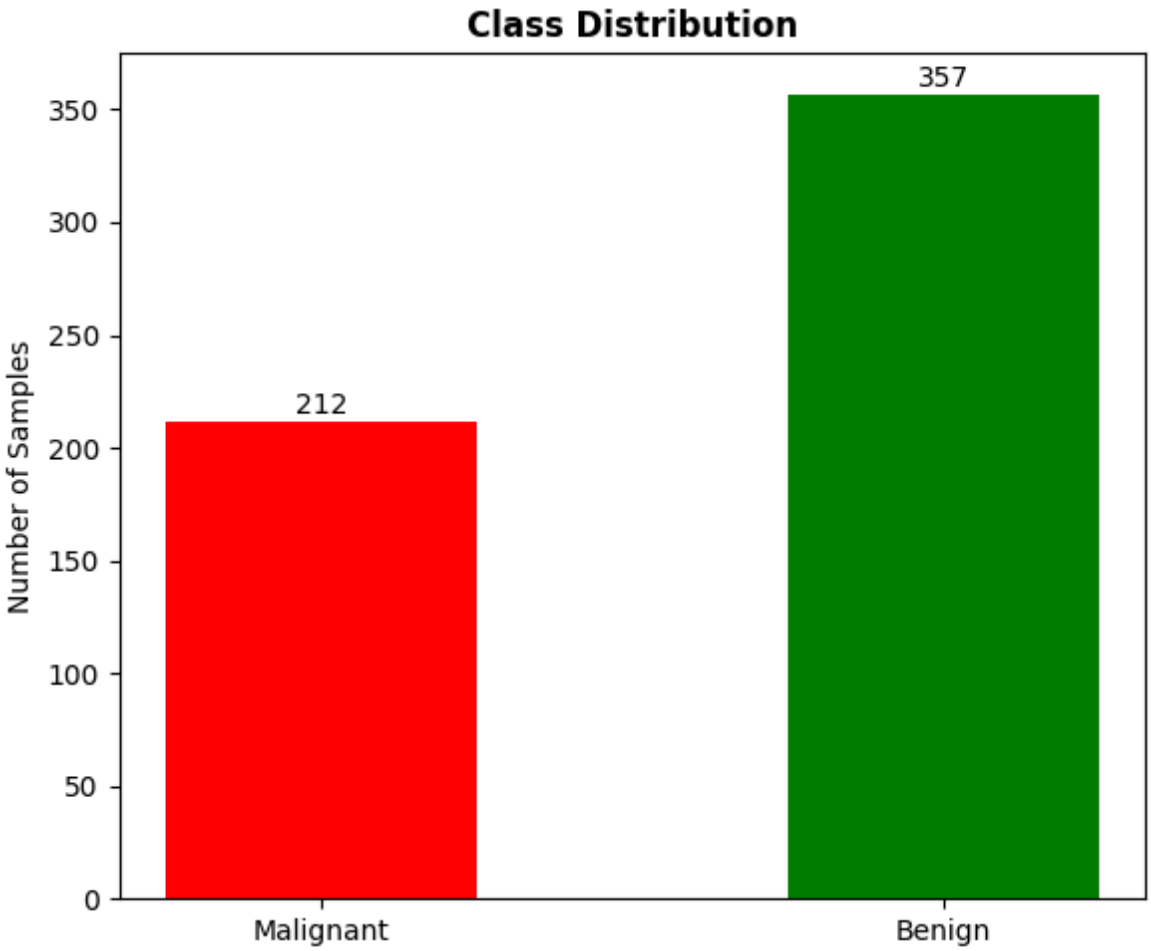
```
Summary statistics for key features:
       mean radius  mean texture  mean perimeter  mean area  mean smoothness  mea
n compactness
count      569.000       569.000         569.000    569.000          569.000
569.000
mean        14.127        19.290          91.969    654.889            0.096
0.104
std          3.524         4.301          24.299    351.914            0.014
0.053
min          6.981         9.710          43.790    143.500            0.053
0.019
25%         11.700        16.170          75.170    420.300            0.086
0.065
50%         13.370        18.840          86.240    551.100            0.096
0.093
75%         15.780        21.800         104.100    782.700            0.105
0.130
max         28.110        39.280         188.500   2501.000            0.163
0.345
```

Plot 1:

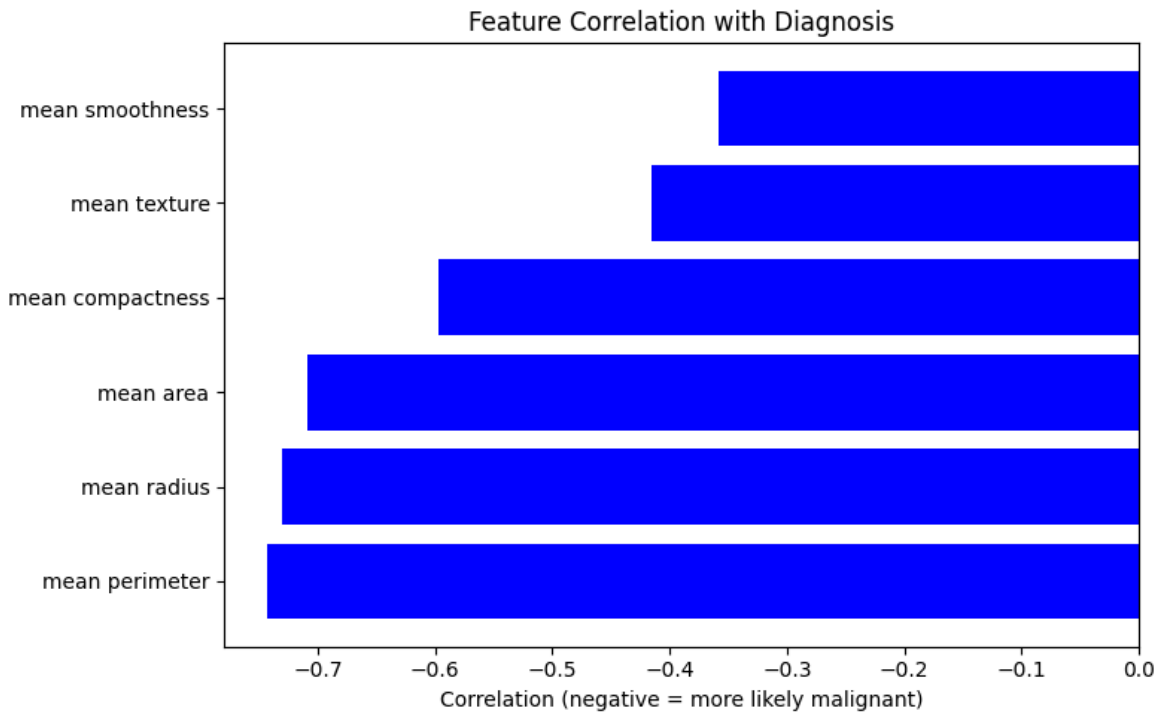
We plot the class distribution and feature correlation first for easy visualisation.

In [65]:
```
fig, axis = plt.subplots(figsize=(6, 5))
counts = df["target"].value_counts()
bars = axis.bar(
```

```
    ["Malignant", "Benign"],
    [counts[0], counts[1]],
    color = ["red","green"],
    width = 0.5,
)
#Adding the count number on top of each bar for clarity
for bar, val in zip(bars, [counts[0], counts[1]]):
    axis.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 3,
        str(val),
        ha = "center",
    )
axis.set_title("Class Distribution", fontweight="bold")
axis.set_ylabel("Number of Samples")
plt.tight_layout()
plt.show()
```



In [66]:
```
fig, axis = plt.subplots(figsize = (8,5))
correlations = df[key_features + ["target"]].corr()["target"].drop("target")
correlations = correlations.sort_values()  #sorting from most negative to most p
axis.barh(correlations.index, correlations.values, color = "blue")
axis.set_title("Feature Correlation with Diagnosis")
axis.set_xlabel("Correlation (negative = more likely malignant)")
plt.tight_layout()
plt.show()
```



Preparing the data for training:

We separate the features(X) from the target (y), where X is all the measurements and y is the diagnosis.

In [67]:
```
X = df[key_features]
y = df["target"]
```

We then split into an 80:20 split for training:test. We use "stratify = y" so that the class proportions are the same in the training and test data. We use "random_state = seed" so we get the same training:test split every time.

In [68]:
```
seed = 3 #We can choose any integer.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state = seed, stratify = y
)
```

Just to make things clear, we print the number of training samples and test samples. As we expect, the ratio is (as close as it gets to) 80:20.

In [69]:
```
print(f"Training samples: {X_train.shape[0]}")
print(f"Test samples: {X_test.shape[0]}")
```

```
Training samples: 455
Test samples: 114
```

We scale the features using StandardScaler(), as the actual units for some features are much larger than others. This means that if they are not scaled, then one feature (e.g. area $\sim$600) will contribute more to the calculation than a different feature (e.g. smoothness $\sim$0.1) in some calculation. By scaling, we remove any unwanted weighting, caused by differing unit sizes. The scaling changes every feature to have 0 mean and 1 standard deviation. [7]

In [70]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Features scaled to zero mean and unit variance.")
```

```
Features scaled to zero mean and unit variance.
```

Training the models:

First, we define our 3 models in a dictionary for easy looping. We use sklearn's in-built models, because this part of the project focuses more on the benefits and pitfalls of different models. For the Logistic Regression, we choose the conventional maximum number of iterations, which is 10,000. For the Random Forest, again we choose the conventional number of trees as 100. [5] And again, for the K-nearest neighbours, we choose the conventional numbers of neighbours as 5. [6] Later on we will discuss why choosing the conventional parameters is a potential pitfall.

We then store these results in the dictionary "results".

In [71]:
```
models = {
    "Logistic Regression": LogisticRegression(max_iter = 10000, random_state = s
    "Random Forest":       RandomForestClassifier(n_estimators = 100, random_sta
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors = 5),
}

results = {}
```

Next, we train the model in two ways.

First, we use cross-validation. This is where we train and test the model on 5 different partitions of the entire training set. So, we train on a separate 16% of the total data each time.This gives us a more reliable estimate than just testing once, although it has less training data.

Then, we train the model using the full training data set.

In [72]:
```
for name, model in models.items():
    print(f"\n--- {name} ---")
    #First we look at cross-validation.
    cv_scores = cross_val_score(
        model, X_train_scaled, y_train, cv = 5, scoring = "accuracy"
    )
    print(f"Cross-val accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f

    #Now we train on the full training set
    model.fit(X_train_scaled, y_train)

    #Making predictions on the test set (data the model has never seen)
    y_pred = model.predict(X_test_scaled)

    #Finally, we calculate the accuracy:
    acc = accuracy_score(y_test, y_pred)
    print(f"Test accuracy:      {acc:.4f}")
```

```
    results[name] = {
        "model": model,
        "cv_mean": cv_scores.mean(),
        "test_accuracy": acc,
        "y_pred": y_pred,
    }
```

```
--- Logistic Regression ---
Cross-val accuracy: 0.9187 (+/- 0.0112)
Test accuracy:      0.9561

--- Random Forest ---
Cross-val accuracy: 0.9143 (+/- 0.0264)
Test accuracy:      0.9474

--- K-Nearest Neighbors ---
Cross-val accuracy: 0.9055 (+/- 0.0256)
Test accuracy:      0.9386
```

Evaluating the models:

We compare the metrics for each model, and output the name of the best model, along with its accuracy scores.

In [73]:
```
best_name = max(results, key=lambda k: results[k]["test_accuracy"])
best_pred = results[best_name]["y_pred"]

print(f"\nBest model: {best_name}")
print(f"Test accuracy: {results[best_name]['test_accuracy']:.4f}")
```

```
Best model: Logistic Regression
Test accuracy: 0.9561
```

We then provide a Classification report for each model, so we can compare different metrics.

Recap (already explained in Random Forest Above):

1. Precision: of all predicted malignant, what proportion actually were?
2. Recall: of all actual malignant, what proportion did we catch?
3. F1: harmonic mean of precision and recall (balanced score)

In [74]:
```
for name, res in results.items():
    print(f"\nClassification Report ({name}):")
    print(classification_report(y_test, res["y_pred"],target_names = data.target
```

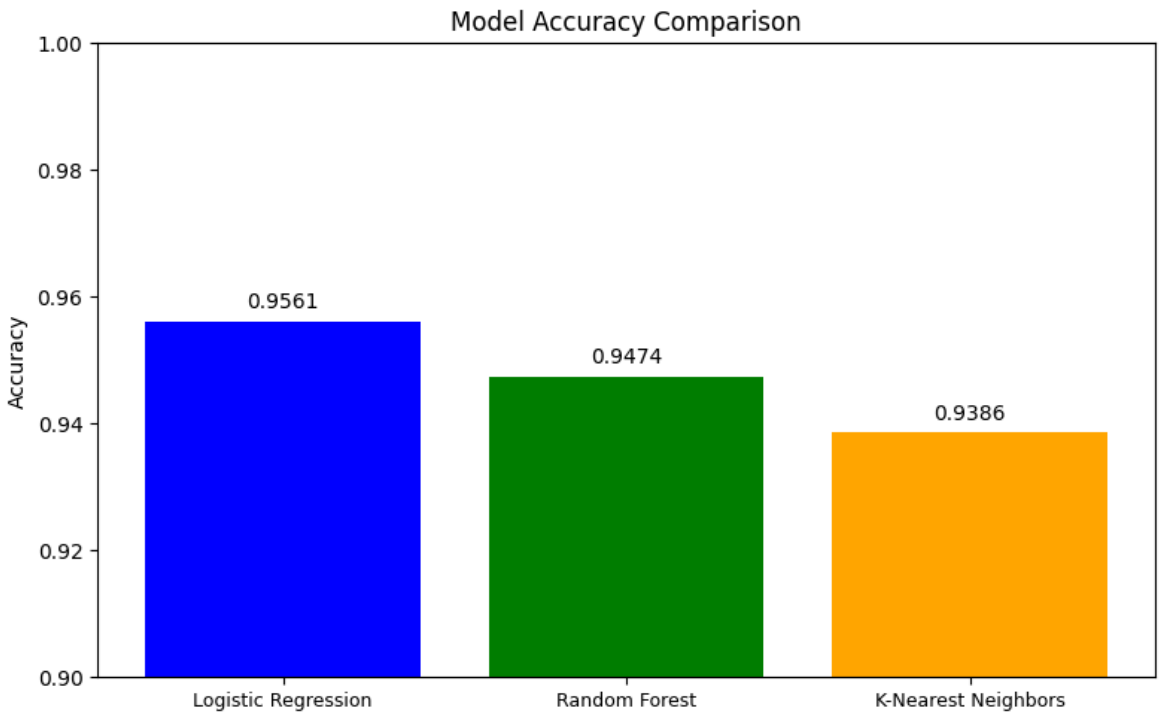
```
Classification Report (Logistic Regression):
              precision    recall  f1-score   support

   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114


Classification Report (Random Forest):
              precision    recall  f1-score   support

   malignant       0.93      0.93      0.93        42
      benign       0.96      0.96      0.96        72

    accuracy                           0.95       114
   macro avg       0.94      0.94      0.94       114
weighted avg       0.95      0.95      0.95       114


Classification Report (K-Nearest Neighbors):
              precision    recall  f1-score   support

   malignant       0.93      0.90      0.92        42
      benign       0.95      0.96      0.95        72

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114
```

We then plot these in ways which make visualising easier.

We plot a graph showing the test accuracy for each model.

We also plot a confusion matrix for each model, showing the number of malignant/benign samples with each model's corresponding predictions.

In [75]:
```
fig, axis = plt.subplots(figsize=(8, 5))

model_names = list(results.keys())
accuracies = [results[n]["test_accuracy"] for n in model_names]
bars = axis.bar(model_names, accuracies, color = ["blue", "green", "orange"])
for bar, acc in zip(bars, accuracies):
    axis.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.002,
        f"{acc:.4f}",
        ha = "center",
    )

axis.set_title("Model Accuracy Comparison")
axis.set_ylabel("Accuracy")
axis.set_ylim(0.9, 1.0)
axis.tick_params(axis = "x", labelsize = 9)
plt.tight_layout()
plt.show()
```



We see that although Logistic Regression was the best model, Random-Forest followed closely behind it, with the K-Nearest Neighbours not being too far away. However, this is likely due to the lack of data. If we had a few thousand instances more, instead of the measly 569, we might learn more about the models. So, from this, we can conclude that Logistic Regression is NOT necessarily the best model in this context (despite the crude statistics suggesting otherwise), rather that no clear conclusion can be made due to the lack of data.

We also chose arbitrary parameters for all 3 models. All the models have such high accuracy rates, which suggests overfitting. Overfitting is where the model follows the training data too closely, and almost predicts the results for the test data as if it were a part of the training data. This is a mistake because the test data could vary vastly to the training data.

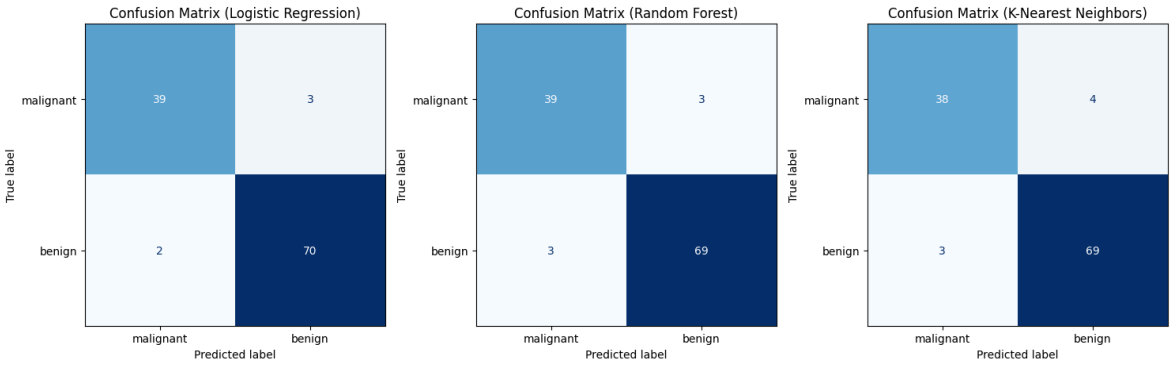
Specifically for each model:

- With Logistic Regression, there may be a case of simply overfitting.
- With the Random Forest, we chose 100 trees but earlier we trimmed our number of features to 6. This is a mistake, because the number of decision trees being about 6 times the number of features means that many of the trees will be very similar. This means we are complicating our model with diminishing returns, so the models takes longer for a similar output and is computationally inefficient. So, for a future data set we would perhaps decrease the number of decision trees, and experiment with the number to find the optimal number such that we get the best otput without overcomplicating our model too much.
- With K-Nearest Neighbours, it may just be that data points with features close to each other, may have 0 meaning in a biological sense, and thus the entire concept of the model is pointless in this context.

However, it is entirely possible (and a few internet searches suggest it is) that breast cancer cells are not too difficult to classify. Another possibility is that the limited quantity data, coupled with the data variation being limited results in easy-to-model data.

However, the main takeaway from this is that simply comparing the raw numbers can be severely misleading. Without considering the underlying reasons (in this case the biology behind Breast Cancer), brute-forcing data can be a big mistake. [4]

In [76]:
```
fig, axes = plt.subplots(1, 3, figsize=(15, 6))

for ax, (name, res) in zip(axes, results.items()):
    cm = confusion_matrix(y_test, res["y_pred"])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=data.targe
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    ax.set_title(f"Confusion Matrix ({name})")

plt.tight_layout()
plt.show()
```



We see none of the models have a quadrant different by more than 1, so there is (likely) not actually a sizeable difference between these models on this dataset. When run multiple times, it is very possible that actually Logistic Regression is not the best model, or maybe even the second-best model.

In summary, a lack of data and a lack of knowledge about the underlying science behind each data point meant that purely comparing statistics for breast cancer cells (or many other real-world applications), could provide a wrong conclusion which would lead us to choose a comparatively weak model.

Reinforcement Learning - Hexapawn

Hexapawn is a game set on a 3x3 chess grid with three opposing pawns on either side moving as standard chess pawns do. The objective of the game is to either:

- Reach the other side of the board
- Leave your opponent with no legal moves
- Capture all your opponents pieces

This is linked to machine learning as it was created to be played against a series of matchboxes (depicting all possible board states) filled with coloured beads (corresponding to a colour coded move). The way it learns is by randomly choosing a

bead from the box and perfomring the move. If this move results in a direct loss, the bead is removed. With enough training rounds and bead removals, this can result in a perfect Hexapawn playing machine that never loses. This is an example of reinfocement learning as it plays a game (performs an action) and adjusts its strategy (ie trial and error). [8]

In [77]:
```
from IPython.display import display
from sympy import Matrix
```

Firstly we set up the basics of the game which include:

- Creating and returning a board: Represented by a 3x3 array with 1 representing white pawns and -1 representing black pawns.
- Returning a list of all possible moves: Returns a 2 element array of the current position of a piece then its new position, represented by a pair.
- Returning the new state of a board after a move is performed: Takes in current board state and move as parameters and returns board state after move.
- Checking for a winner by win criteria specified above

In [78]:
```
#Basics of the Game

def initialize_board():
    return np.array([[-1,-1,-1],
                     [0,0,0],
                     [1,1,1]])

def get_moves(board, player):
    moves = []

    for row in range(0,3):
        for col in range(0,3):
            if (board[row,col] == player):

                #Check new row is valid
                new_row = row - player
                if (0 <= new_row <= 2):

                    #Forward Moves
                    if (board[new_row,col] == 0):
                        moves.append(((row,col), (new_row,col)))

                    #Diagonal takes
                    for diag in [-1,1]:
                        diag_col = col + diag
                        if(0 <= diag_col <= 2 and board[new_row,diag_col] == -pl
                            moves.append(((row,col), (new_row, diag_col)))

    return(moves)

def perform_move(board, move):
    new_board = board.copy()
    (row1,col1),(row2,col2) = move
    new_board[row2,col2] = board[row1,col1]
    new_board[row1,col1] = 0
    return(new_board)
```

```
def check_winner(board, player): #Returns player number of winner

    #Check for no pawns left
    if not(np.any(board == -1)):
        return 1
    if not(np.any(board == 1)):
        return -1

    #Check for reaching opposite side of board
    if 1 in board[0]:
        return 1
    if -1 in board[2]:
        return -1

    #Check for no possible moves for opponent
    if (len(get_moves(board, player)) == 0):
        return(-player)

    return 0
```

We then define the machine as a class [9] (this is done to allow for 2 machines to face each other later) with the following routines:

- **init:** creates a self variable/dictionary called boxes which works as the set of matchboxes. Also created the self constant INITIAL BEADS as we were planning on exploring that effect on the model.
- **print_box:** this will print the state of the box dictionary (ie print all possible moves and how many beads it has left)
- **get_box:** returns the "matchbox" corresponding to the current boardstate (or initialise one if necessary)
- **choose_move:** randomly selects a bead and returns the move corresponding to it.
- **reinforce:** updates boxes depending on loss or win (reward) and removes or adds a bead respectively (only adds a bead if constant ADD_BEAD is True)

In [79]:
```
class Machine: #Define a class to allow machines to face each other
    def __init__(self, initial_beads=1):
        # Dictionary works like: state -> {move -> {bead_count}}
        self.boxes = {}
        self.INITIAL_BEADS = initial_beads

    def print_box(self):
        print("Boxes created: ",len(self.boxes))
        for state,contents in self.boxes.items():
            display(Matrix(state).reshape(3,3))
            for move,bead in contents.items():
                print(move, bead)
            print("")

    def get_box(self, board, moves):
        state = tuple(board.reshape(1,9)[0])
        # Cannot have arrays being key as they are unshashable
        # Reshaping board and converting to tuple corrects this

        if state not in self.boxes:
            # Initialize unseen matchboxes
```

```
            state_dict = {}
            for move in moves:
                state_dict[move] = self.INITIAL_BEADS #Initializes each possible
            self.boxes[state] = state_dict

        return self.boxes[state]

    def choose_move(self, board, moves):
        box = self.get_box(board, moves)

        valid_moves = []
        for move in moves:
            if box[move] > 0: # Ignores moves that have 0 beads
                for bead in range(0, box[move]): #Adds a copy of the move per be
                    valid_moves.append(move)

        if len(valid_moves) == 0:
            return moves[np.random.randint(0,len(moves))] #Chooses a random if n

        return valid_moves[np.random.randint(0,len(valid_moves))] #Returns a ran

    def reinforce(self, last_move, reward, ADD_BEAD):
        state = last_move[0]
        move = last_move[1]

        if ((reward > 0) and (ADD_BEAD)):
            self.boxes[state][move] += 1 # Adds a bead for a win

        if reward < 0:
            self.boxes[state][move] -= 1 # Removes a bead for a loss

            if self.boxes[state][move] < 0:
                self.boxes[state][move] = 0 # Sets beads to 0 if negative beads
```

In [80]:

```
def player_move(board, moves):
    for i in range(0, len(moves)):
        option_board = board.copy()
        print(i)
        display(Matrix(perform_move(option_board, moves[i])))
    move_choice = int(input("\n"))
    return(moves[move_choice])
```

Now we train the model. This routine takes in many differing parameters to allow for ease of changing the model:

- **MACHINE_PLAYER:** Player number of the machine
- **RANDOM:** True if the machine is trained on Random inputs False to be trained on human games
- **ADD_BEAD:** True if we add a bead for a win, False otherwise.
- **MATCHES:** Number of games trained on.
- **INITIAL_BEADS:** Number of starting beads per box.

In [81]:

```
def train_machine(MACHINE_PLAYER, RANDOM, ADD_BEAD, MATCHES=20, INITIAL_BEADS =
    machine = Machine(INITIAL_BEADS)
    results = []

    for _ in range(MATCHES):
```

```
        board = initialize_board()
        player = 1
        history = []
        last_move = (0,0)

        while (check_winner(board,player) == 0):
            moves = get_moves(board, player)

            if player == MACHINE_PLAYER:
                move = machine.choose_move(board, moves)
                last_move = (tuple(board.reshape(1,9)[0]), move) #Saves the most

            else:
                if RANDOM:
                    move =  moves[np.random.randint(0,len(moves))]

                else:
                    move = player_move(board, moves) #For human training

            board = perform_move(board, move)

            player *= -1

        reward = check_winner(board, player) * MACHINE_PLAYER #Corrects player n
        machine.reinforce(last_move, reward, ADD_BEAD)
        last_move = (0,0)
        results.append(reward)

    return machine, results
```

In [82]:

```
def display_results(results, machine, PRINT_BEADS = False):
    win_count = 0
    loss_count = 0

    for result_val in results:
        if result_val == 1:
            win_count +=1
        else:
            loss_count += 1

    print(win_count, loss_count)
    print("Win rate: ", win_count/len(results))


    plt.plot(results)
    plt.title("Hexapawn")
    plt.xlabel("Games")
    plt.ylabel("Reward")
    plt.show()

    if PRINT_BEADS:
        machine.print_box()
```

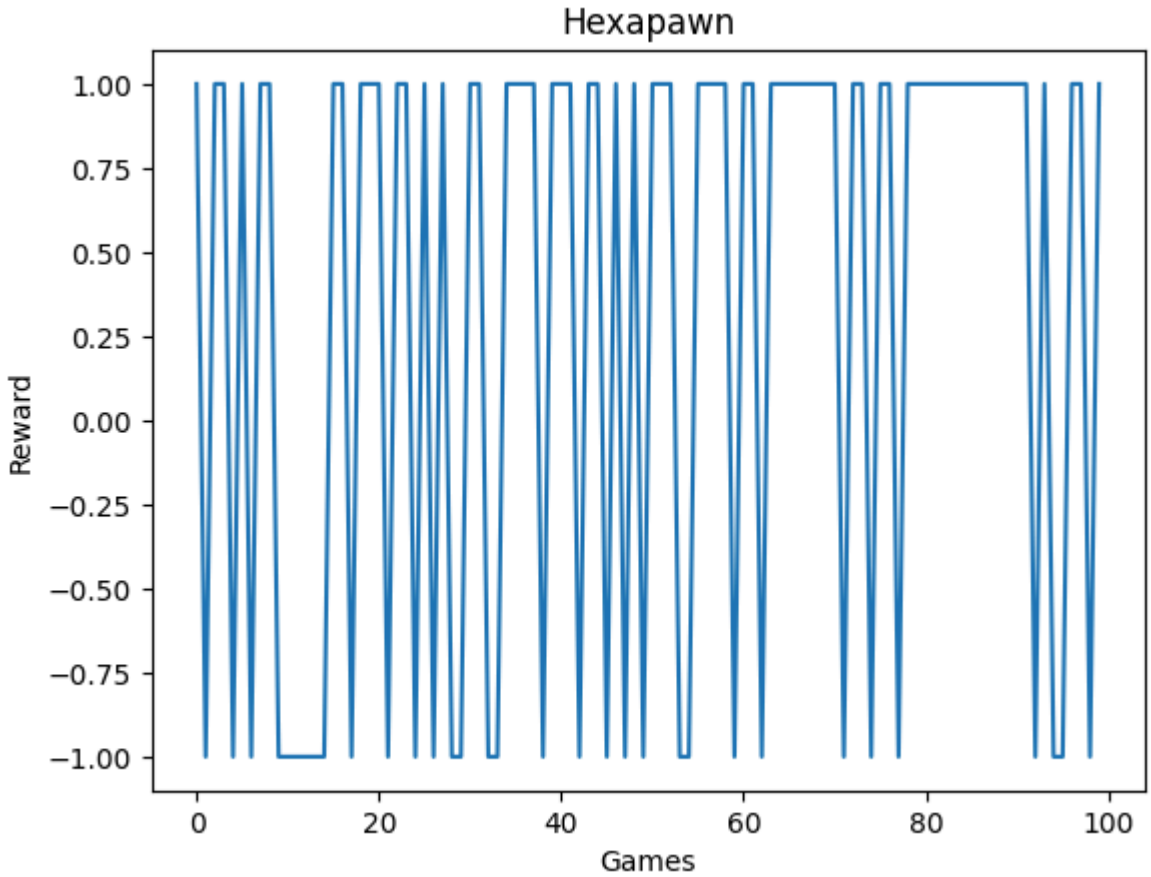
Now we create 2 machines representing white and black pawns on 100 matches then print the training results (1 for win, -1 for loss)

In [83]:

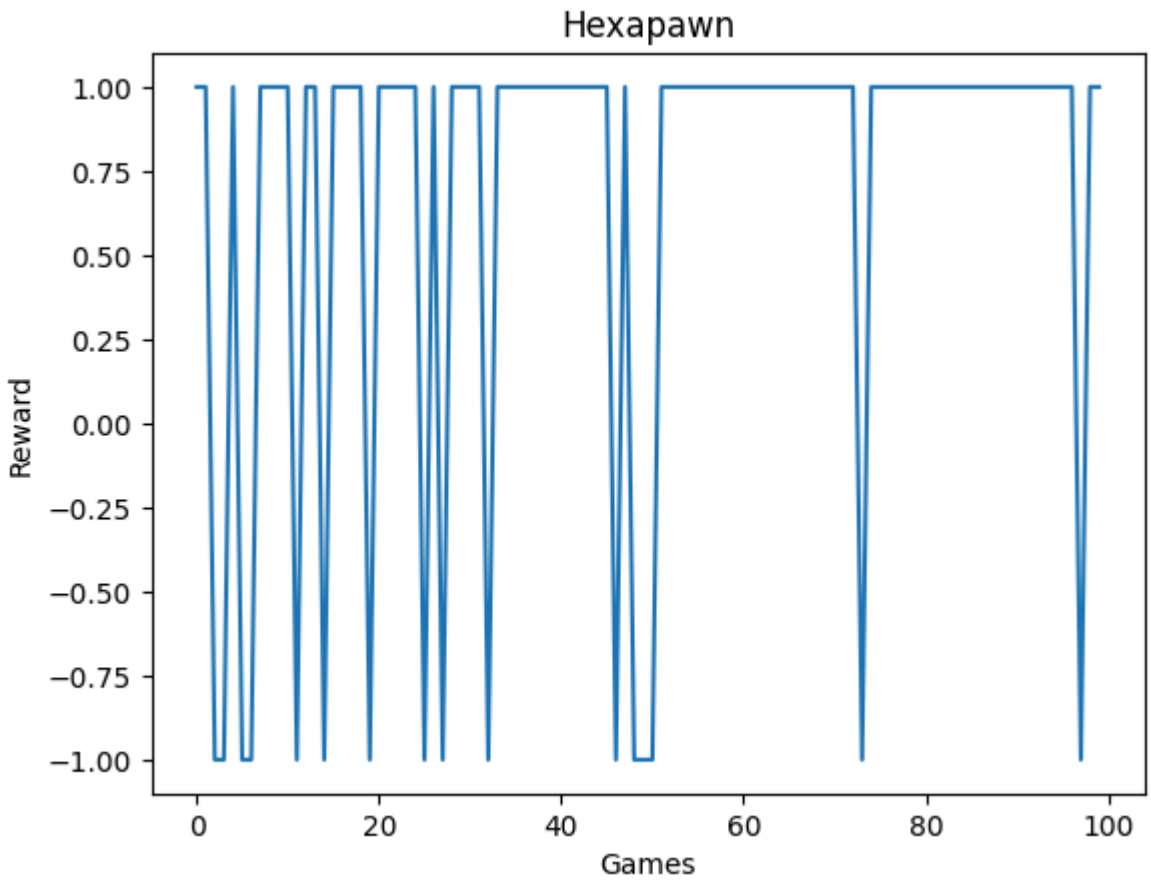
```
RANDOM = True
ADD_BEAD = False
NUM_ITER = 100
```

```
INITIAL_BEADS = 1
PRINT_BEADS = False # If we want to print the state of the boxes (number of bead

machine1, results1 = train_machine(1, RANDOM, ADD_BEAD, NUM_ITER, INITIAL_BEADS)
print("White Results")
display_results(results1, machine1, PRINT_BEADS)

machine2, results2 = train_machine(-1, RANDOM, ADD_BEAD, NUM_ITER, INITIAL_BEADS
print("Black Results")
display_results(results2, machine2, PRINT_BEADS)
```

```
White Results
67 33
Win rate:  0.67
```




```
Black Results
84 16
Win rate:  0.84
```



We then made the function for a test match between a machine and either a random or human opponent

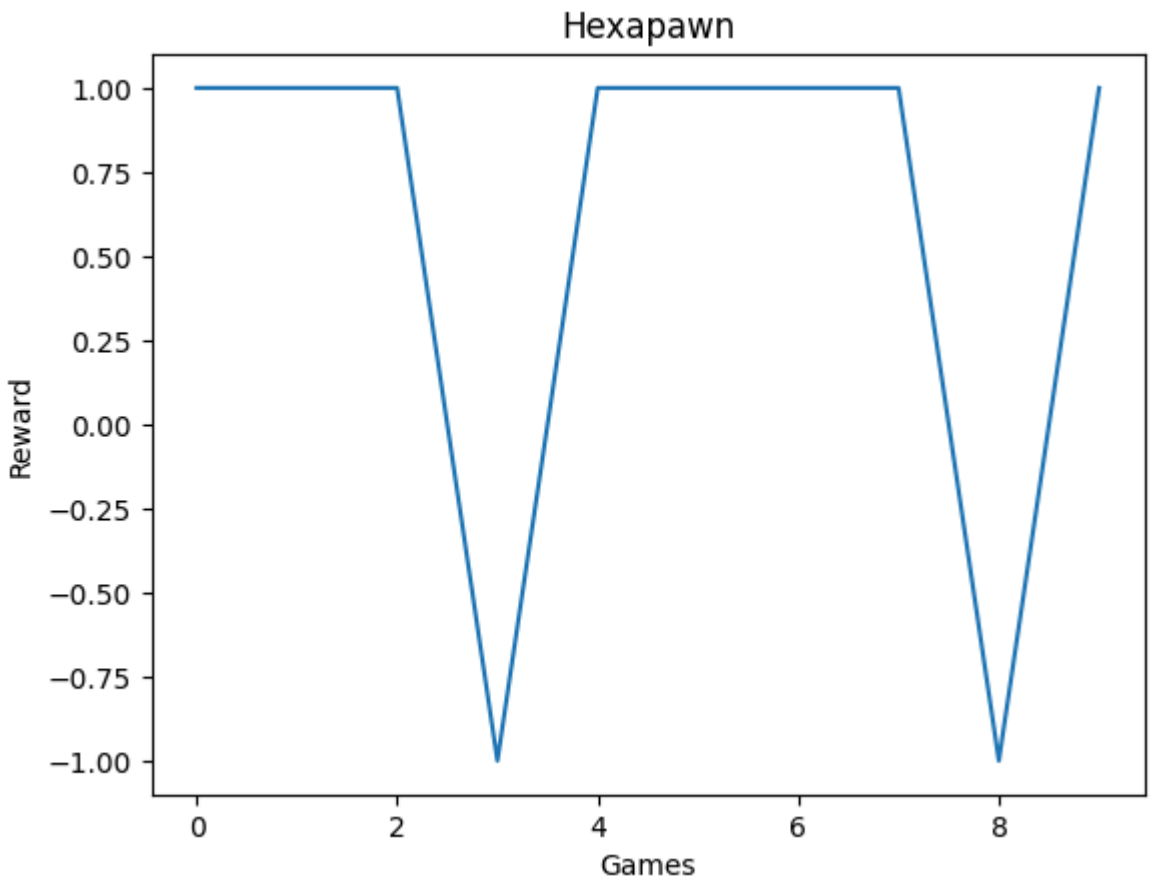
In [84]:
```
def test_match(machine, MACHINE_SIDE, RANDOM = True, PRINTGAME = False):
    board = initialize_board()
    player = 1
    DISPLAY = (RANDOM and PRINTGAME) or not(RANDOM) #Displays game if (random an

    while (check_winner(board, player) == 0):
        moves = get_moves(board,player)

        if (player == MACHINE_SIDE):
            move = machine.choose_move(board, moves)
            if DISPLAY:
                print("Machine Chooses")

        if (player == -MACHINE_SIDE):
            if RANDOM:
                move = moves[np.random.randint(0,len(moves))]
                if DISPLAY:
                    print("Random Chooses")
            else:
                move = player_move(board,moves)
                print("Player Chooses")

        board = perform_move(board, move)
        player *= -1

        if DISPLAY:
            display(Matrix(board))
            print("")
```

```
    if DISPLAY:
        print("Winner is player", -player) #Not the winners turn so -player is w

    return(-player * MACHINE_SIDE) #Returns win or loss from machine perspective
```

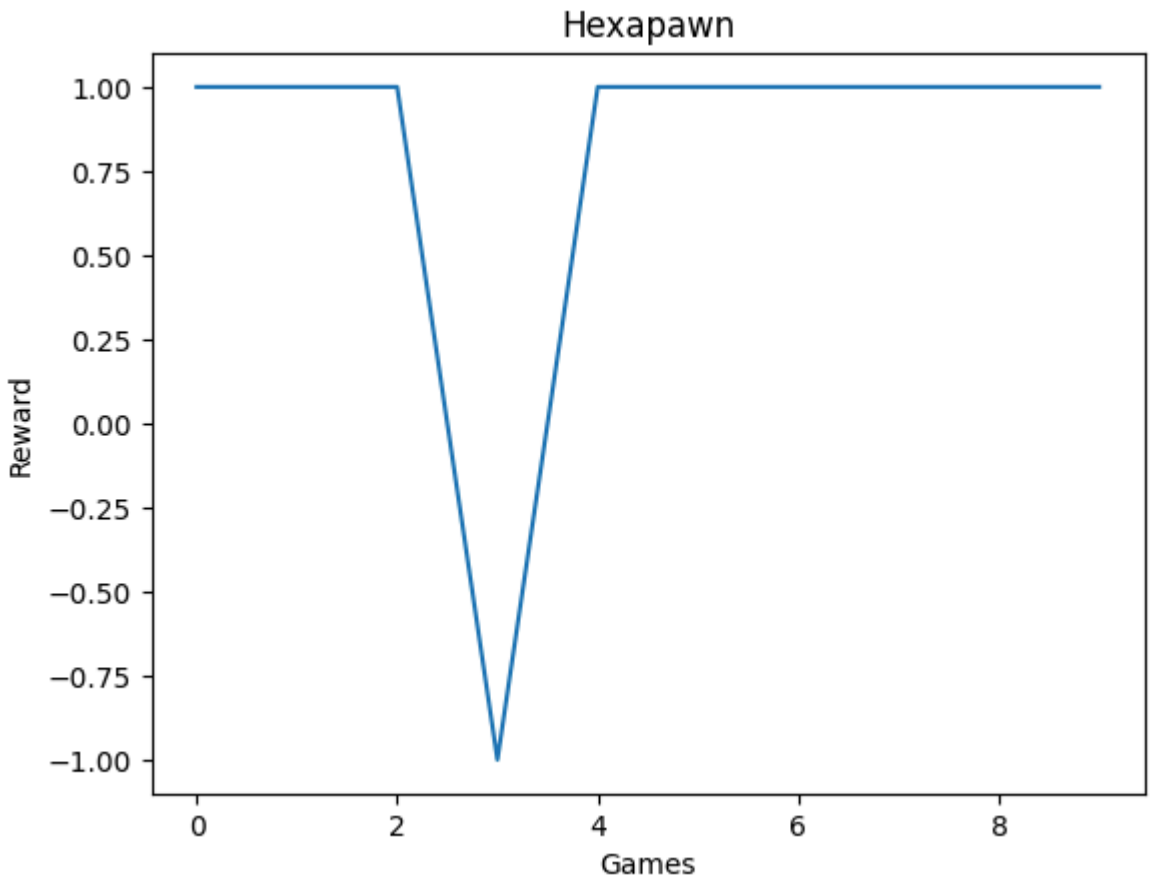
We then tested the models to see how they do after training

In [85]:
```
#White
test_results1 =[]
for game in range(10):
    test_results1.append(test_match(machine1, 1, True, False))
display_results(test_results1, machine1)

#Black
test_results2 = []
for game in range(10):
    test_results2.append(test_match(machine2, -1, True, False))
display_results(test_results2, machine2)
```

```
8 2
Win rate:  0.8
```




```
9 1
Win rate:  0.9
```



These results usually give a 70-90% success rate for white and consistent 90-100% for Black. This has been proven with game theory that black can get perfect play whereas white cannot.

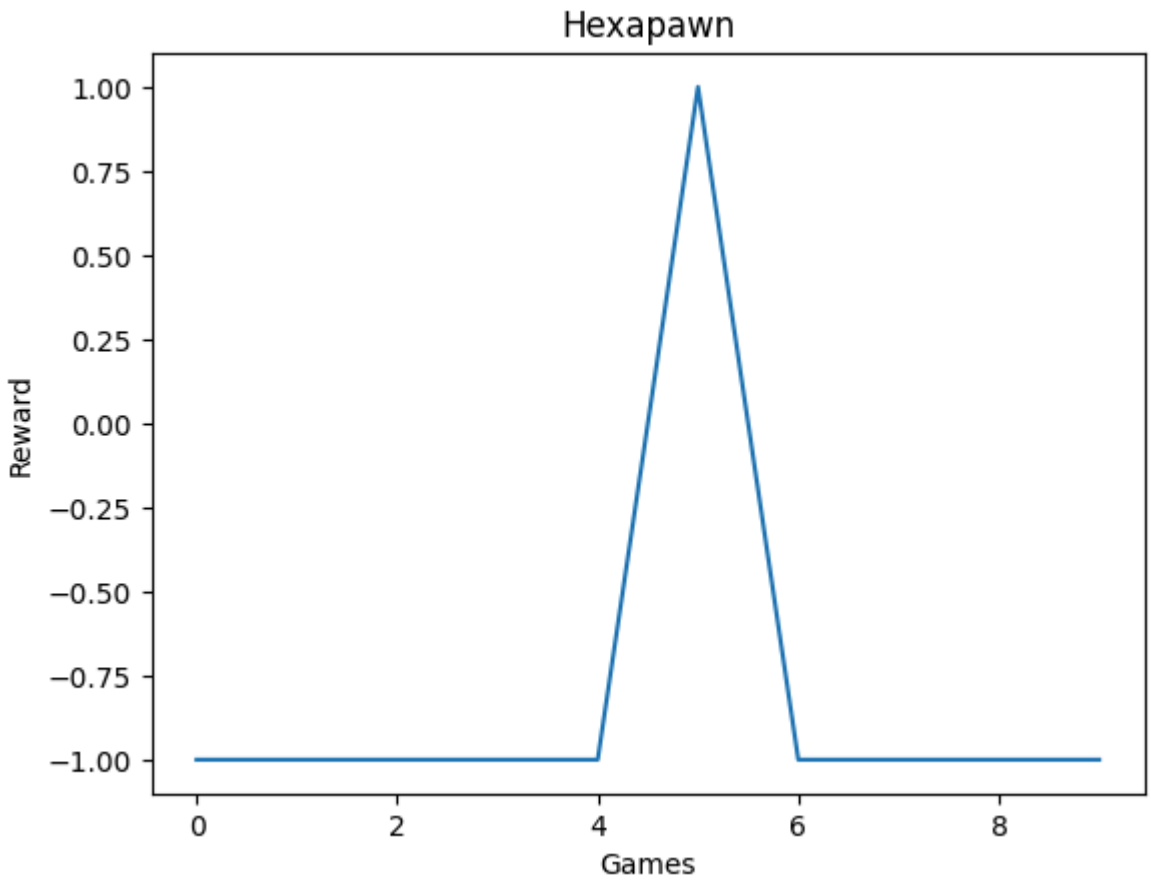
We then created a routine for 2 machines to play each other (this is why we implemented them as classes)

In [86]:
```
def machine_match(machine1, machine_1, PRINTGAME = False):
    board = initialize_board()
    player = 1

    while (check_winner(board, player) == 0):
        moves = get_moves(board,player)

        if (player == 1):
            move = machine1.choose_move(board, moves)
            if PRINTGAME:
                print("Machine 1 Chooses")

        if (player == -1):
            move = machine_1.choose_move(board, moves)
            if PRINTGAME:
                print("Machine 2 Chooses")

        board = perform_move(board, move)
        player *= -1

        if PRINTGAME:
            display(Matrix(board))
            print("")
```

```
    print("Winner is machine", -player) #Not the winners turn so -player is winn
    return(-player) #Returns win or loss from machine perspective
```

In [87]:
```
test_results = []
for game in range(10):
    test_results.append(machine_match(machine1, machine2))

display_results(test_results, machine1)
```

```
Winner is machine -1
Winner is machine -1
Winner is machine -1
Winner is machine -1
Winner is machine -1
Winner is machine 1
Winner is machine -1
Winner is machine -1
Winner is machine -1
Winner is machine -1
1 9
Win rate:  0.1
```



The results are given from the perspective of white (1 for white win, -1 for black win) and again we see consistent 90-100% win rate for black.

Deep Neural Network

This is a model that explores deep neural networks.

A deep neural network is a network with many hidden layers. It is a concept that is an extension of A4.

Here, we will first discuss the more abstract concept and benefit of hidden layers, and then move on to the actual code demonstration using the poker hand data set.

The hidden layer consists of the number of neurons, the number of layers, and the number of neurons in each layer. These are the determinants of the deep neural network, and are the neuron architecture. The number of neurons (width) determines the number of features that a layer can represent simultaneously. The number of hidden layers (depth) acts like a composition function, receiving the output from the previous layer and learn the data from there rather than from the initial input. The last property is the number of neurons in each layer. We will be using a model such that the number of neurons in each layer decreases (e.g. (64, 32, 16,8) being 64 neurons in the first layer, 32 neurons in the second layer, and so on). This is called a funnel pattern, and by using this architecture, the lower layers learn the raw data and the higher layers learn more abstract features, which prevents neurons from learning similar features over and over. By this design, it also decreases the training cost and prevents overfitting, with the decreased number of total parameters compared to using a neuron architecture with a consistent width pattern such as (64,64,64,64).

Model Overview

The model we will be building is a **classification model** , and we will ultimately be estimating the accuracy of the model's classification.

Dataset: Poker Hand Dataset from the UCI Machine Learning Repository

Independent variable: neural network architecture (the number of hidden layers, number of neurons)

Dependent variable: accuracy of the classification

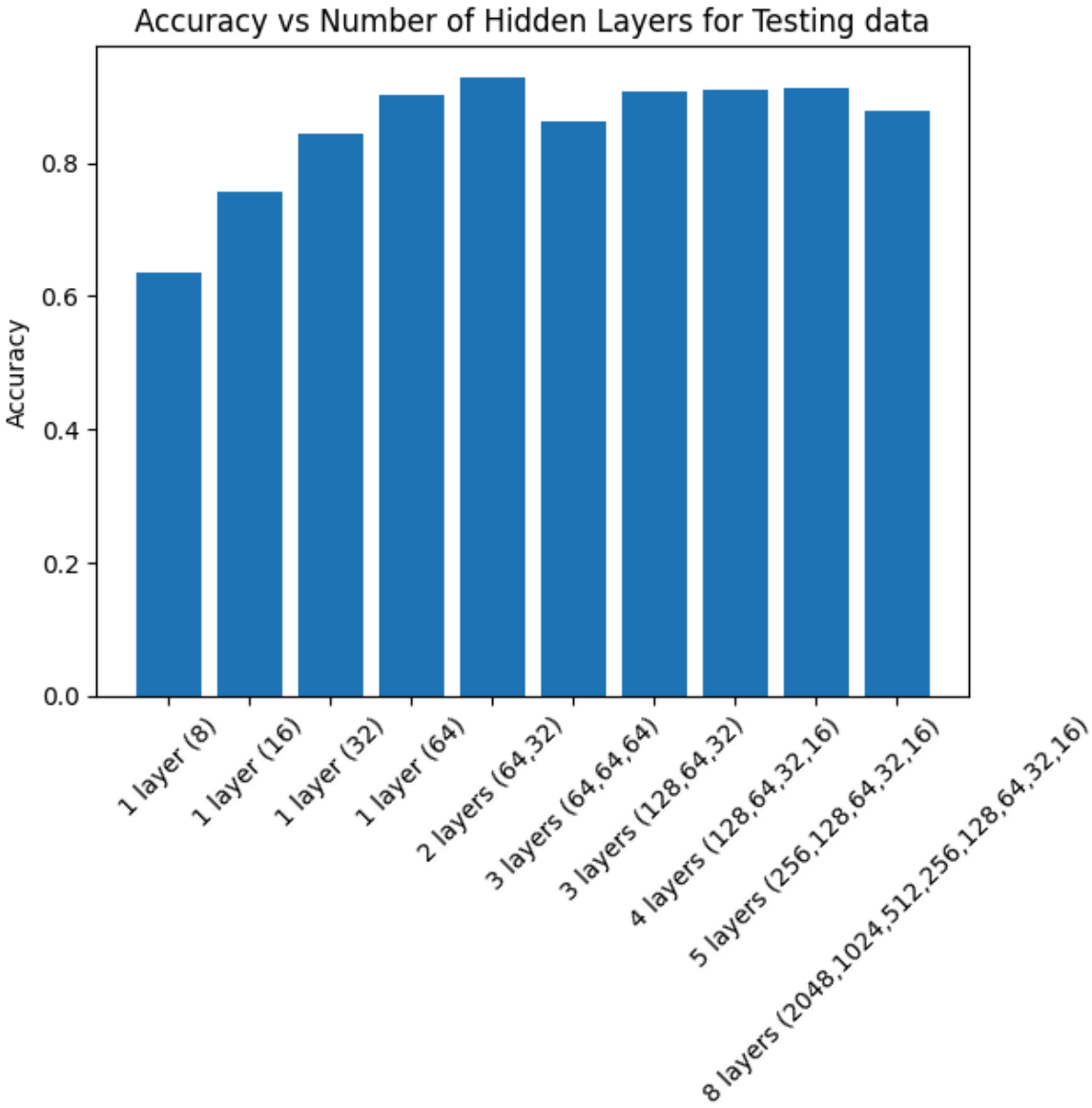
The information for the actual codes is commented alongside the codes.

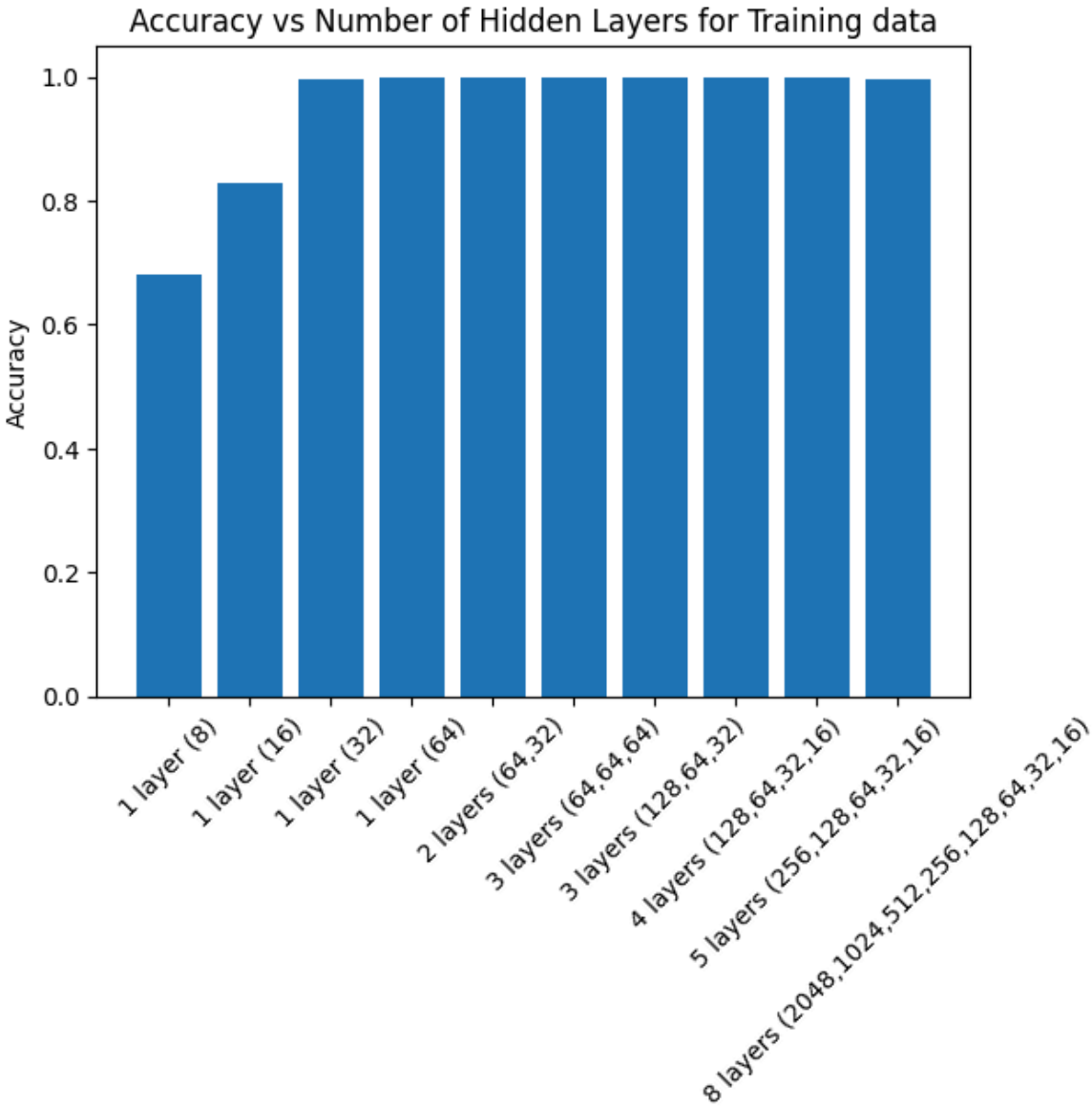
In [88]:
```
#importing needed libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# load dataset using pandas
data = pd.read_csv("poker-hand-training-true.data", header=None,nrows=13000)

# Separating the data into input and output
# input being the cards and the output being the target label
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# dividing the columns into the numbers for the card's suit and rank.
# the data is stored like [1st card's suit][1st card's rank][2nd card's suit]..[
# all represented by numbers.
```

```
suit_col = [0,2,4,6,8]
rank_col = [1,3,5,7,9]

#OneHotEncoder returns the categories into binary numbers, so that the model can
rank_encoder = OneHotEncoder(sparse_output=False)
X_rank_encoded = rank_encoder.fit_transform(X[rank_col])

encoder = OneHotEncoder(sparse_output=False)
X_suit_encoded = encoder.fit_transform(X[suit_col])

#Combining the suit and the rank
X_processed = np.hstack([X_suit_encoded, X_rank_encoded])

# train-test split (declaring test dataset size
# using train_test_split from sklearn
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=40

# Creating models with different architectures and labeling them accordingly.
architectures = {
    "1 layer (8)": (8),
    "1 layer (16)": (16),
    "1 layer (32)": (32),
    "1 layer (64)": (64,),
    "2 layers (64,32)": (64, 32),
    "3 layers (64,64,64)": (64,64,64),
    "3 layers (128,64,32)": (128, 64, 32),
    "4 layers (128,64,32,16)": (128, 64, 32, 16),
    "5 layers (256,128,64,32,16)": (256,128,64,32,16),
    "8 layers (2048,1024,512,256,128,64,32,16)": (2048,1024,512,256,128,64,32,16
}

results1 = {}
results2 = {}

# Implementing machine learning from SKLearn.
for name, structure in architectures.items():
    mlp = MLPClassifier(

        #here, implementing the hidden layer neural architecture
        hidden_layer_sizes=structure,

        #adding non-linearity
        activation='relu',

        #optimization algorithm
        solver='adam',

        # Max iteration for this one is very high, since we are using the same d
        # We are making sure that the 1-layer models run to the end as well.
        max_iter=2000,
        random_state=42
    )

    mlp.fit(X_train, y_train)

    # Storing output to each of the y values
    y_train_pred = mlp.predict(X_train)
    train_acc = accuracy_score(y_train, y_train_pred)

    y_test_pred = mlp.predict(X_test)
```

```
    test_acc = accuracy_score(y_test, y_test_pred)

    # Storing results
    results1[name] = test_acc
    results2[name] = train_acc

    print(f"{name} -> Train Accuracy:{train_acc:.4f}, Test Accuracy:{test_acc:.4

# Plot results
# graphical output for the testing dataset
plt.figure()
plt.bar(results1.keys(), results1.values())
plt.xticks(rotation=45)
plt.ylabel("Accuracy")
plt.title("Accuracy vs Number of Hidden Layers for Testing data")

# graphical output for training dataset
plt.figure()
plt.bar(results2.keys(), results2.values())
plt.xticks(rotation=45)
plt.ylabel("Accuracy")
plt.title("Accuracy vs Number of Hidden Layers for Training data")

plt.show()
```

```
1 layer (8) -> Train Accuracy:0.6797, Test Accuracy:0.6340
1 layer (16) -> Train Accuracy:0.8279, Test Accuracy:0.7562
1 layer (32) -> Train Accuracy:0.9953, Test Accuracy:0.8442
1 layer (64) -> Train Accuracy:1.0000, Test Accuracy:0.9030
2 layers (64,32) -> Train Accuracy:1.0000, Test Accuracy:0.9290
3 layers (64,64,64) -> Train Accuracy:1.0000, Test Accuracy:0.8630
3 layers (128,64,32) -> Train Accuracy:1.0000, Test Accuracy:0.9077
4 layers (128,64,32,16) -> Train Accuracy:0.9999, Test Accuracy:0.9107
5 layers (256,128,64,32,16) -> Train Accuracy:0.9993, Test Accuracy:0.9130
8 layers (2048,1024,512,256,128,64,32,16) -> Train Accuracy:0.9959, Test Accurac
y:0.8775
```


Accuracy vs Number of Hidden Layers for Testing data
Accuracy
0.0
0.2
0.4
0.6
0.8
1 layer (8)
1 layer (16)
1 layer (32)
1 layer (64)
2 layers (64,32)
3 layers (64,64,64)
3 layers (128,64,32)
4 layers (128,64,32,16)
5 layers (256,128,64,32,16)
8 layers (2048,1024,512,256,128,64,32,16)



Outcome

Overall, the training dataset gives better accuracy, and they almost converge to 100% accuracy as the number of hidden layers increases.

Looking at the testing dataset results.

Number of Neurons The accuracy increases as the number of neurons increases. This is seen from the comparison between the 1-layer models.

Number of hidden layers For the number of hidden layers, initially, the accuracy increases as the number of layers increases, but it then stops, reaching a plateau. We can also see that 3 layers with a consistent number of neurons (64,64,64) have a lower accuracy compared to the 3 layers that implement a funnel pattern (128,64,32).

Model Evaluation

Here, we will explore the concept of overfitting and underfitting and how it is shown in the model.

Underfitting This happens when a model's design and architecture are too simple to completely analyze a complicated dataset. For this model, it's happening when there are too few neurons and hidden layers, causing low accuracy, such as the model with 1-layer having around 65% accuracy.

Overfitting This happens when the model's design and architecture are too complicated compared to the simple data it is processing. By having so many parameters to process data, the model memorizes all the data instead of learning and generalizing a pattern out of it, causing a low test accuracy. This is shown for the 8-layer model in the demonstration, where the training data gives an almost 100% accuracy, whereas the test data gives it a below 90% accuracy.

References

Datasets used

- [1] https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
- [2] https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset
- [3] https://archive.ics.uci.edu/dataset/158/poker+hand

References

- [4] https://www.w3schools.com/python/python_ml_confusion_matrix.asp
- [5] https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/
- [6] https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbor-algorithm-in-python/
- [7] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
- [8] https://www.cs.williams.edu/~freund/cs136-073/GardnerHexapawn.pdf
- [9] https://www.w3schools.com/python/python_class_properties.asp
- MA124 Mathematics by Computer Lessons and Files
- Previous Knowledge

Notes about this submission

You will submit a single Jupyter notebook for this project assignment. Details will be provided for this on the MA124 Moodle page.

- The last thing you should do before submitting the notebook is to Restart Kernel and Run All Cells. You should then save the notebook and submit the .ipynb file. **You**

will lose marks if you submit a notebook that has not been run.

- You are expected to add code and markdown cells to this document as appropriate to provide your responses to the tasks. Instructions about this are given throughout but feel free to add markdown cells at any point to provide clarity or comments.

- Likewise, to help the reader, please provide appropriate comments in your code (for example functions or blocks of code should have comments about what they do, variables should be described in comments, as appropriate).